

A knowledge-guided agentic framework for mitigating patient-context ambiguity in health queries

Mahyar Abbasian, Saba A. Farahani, Arshia Ilaty, Hung Cao, Ramesh Jain, Amir M. Rahmani

January 2026

Abstract

Patients often submit short, underspecified queries to healthcare chatbots that lack the patient-specific information needed to determine an appropriate response. Although these queries may be linguistically clear, they can support multiple plausible answers depending on undisclosed factors such as symptoms, diagnoses, medications, allergies, or dietary restrictions. A language model answering such a query directly may therefore rely on unsupported assumptions about the patient. We introduce a knowledge-guided agentic framework for mitigating patient-context ambiguity before final response generation. The framework operates between the patient and an otherwise unchanged downstream language model. It interprets the initial query, uses a task-specific knowledge graph to construct a set of plausible hypotheses, identifies the missing patient-context variables needed to distinguish among them, and asks targeted follow-up questions. The original query and the acquired context are then combined into a clarified prompt for the downstream model. We evaluated the framework across five language models using two controlled ambiguity-mitigation benchmarks: diagnosis retrieval from 1,034 symptom queries with clinically relevant evidence systematically masked, and dietary-safety classification from 487 queries with decisive health context omitted. The framework was compared with direct answering of the underspecified query and with rephrasing the same query without acquiring new patient information. In diagnosis retrieval, it increased overall exact Top-1 accuracy by at least 57.1 percentage points and selective exact Recall@5 by at least 77.7 percentage points across the five evaluated models compared with direct prompting. In dietary-safety classification, it improved accuracy across all five models and achieved the highest Matthews correlation coefficient for four. In repeated-generation analyses, the clarified prompts also reduced the predictive uncertainty of the downstream language models compared with the original underspecified queries. These findings show that an intermediary agent that actively elicits missing patient context can improve the accuracy and predictive consistency of downstream language-model responses without modifying or fine-tuning the final model.

1 Introduction

Patients increasingly use general-purpose healthcare chatbots to seek medical information and guidance. Unlike structured clinical encounters, these interactions often begin with short, open-ended questions written in everyday language and without a complete medical history. Recent physician-led evaluation of patient-posed medical questions found that publicly available chatbots frequently produced problematic responses, with missing information and failure to obtain an appropriate history among the recurring concerns [1]. The challenge is not limited to whether a language model possesses sufficient medical knowledge. A patient may ask, for example, “What should I take for a headache?” The question is linguistically clear, but the

Table 1: **Illustrative effect of patient-specific context on an otherwise unchanged health query.** The examples are conceptual and show why the same linguistically clear query may require different response strategies as additional patient context becomes available. They are not intended as treatment recommendations or as outputs from the evaluated models.

Patient query	Patient-specific context	Implication for the response
What should I take for a headache?	Not provided	Several common options may appear plausible, but the information needed to distinguish among them is absent.
What should I take for a headache?	Pregnancy	The response must account for pregnancy-related medication constraints and may differ from general over-the-counter guidance.
What should I take for a headache?	Use of an anticoagulant	The response must consider medication interactions and bleeding risk before suggesting an option.
What should I take for a headache?	Liver disease	The response must consider whether common medications are appropriate given impaired liver function.
What should I take for a headache?	Multiple relevant conditions or warning signs	A specific medication recommendation may be inappropriate, and the response may instead require escalation or professional assessment.

appropriate response may depend on information that has not been disclosed, such as pregnancy, anticoagulant use, liver disease, allergies, symptom severity, or accompanying warning signs. Without this context, several answers may be plausible for different patients, while no single answer is adequately supported for the patient asking the question.

Table 1 illustrates this problem. The surface form of the query remains unchanged, but the information needed to formulate an appropriate response changes as patient-specific context is introduced. The ambiguity therefore does not necessarily arise from unclear wording. It arises because the query does not contain the personal information needed to distinguish among multiple context-dependent responses.

This problem represents a distinct form of ambiguity. Linguistic ambiguity occurs when the wording of a query permits multiple interpretations [2]. Knowledge ambiguity occurs when resolving the query requires external factual information that is unavailable to the model [3–7]. Intent ambiguity occurs when the user’s underlying goal is unclear [8, 9]. These forms may be addressed through paraphrasing, external retrieval, factual grounding, or clarification of the intended task. By contrast, *patient-context ambiguity* occurs when the query is understandable and the relevant general medical knowledge may already be available, but the appropriate response depends on a patient-specific variable that has not been disclosed. This variable may reside only with the patient and therefore cannot be recovered from the language model’s parameters or from an external corpus. Resolving this form of ambiguity requires acquiring new information from the user rather than inferring one answer from the original query.

This distinction can also be understood in terms of the knowledge available to a healthcare chatbot. First, a language model draws on *parametric knowledge* encoded during pretraining, instruction tuning, or domain adaptation. Second, it may draw on *external knowledge*, including clinical guidelines, biomedical literature, drug resources, and structured ontologies, through retrieval or knowledge-graph grounding [10–12]. Third, an appropriate response may require *patient-specific contextual knowledge*, including current

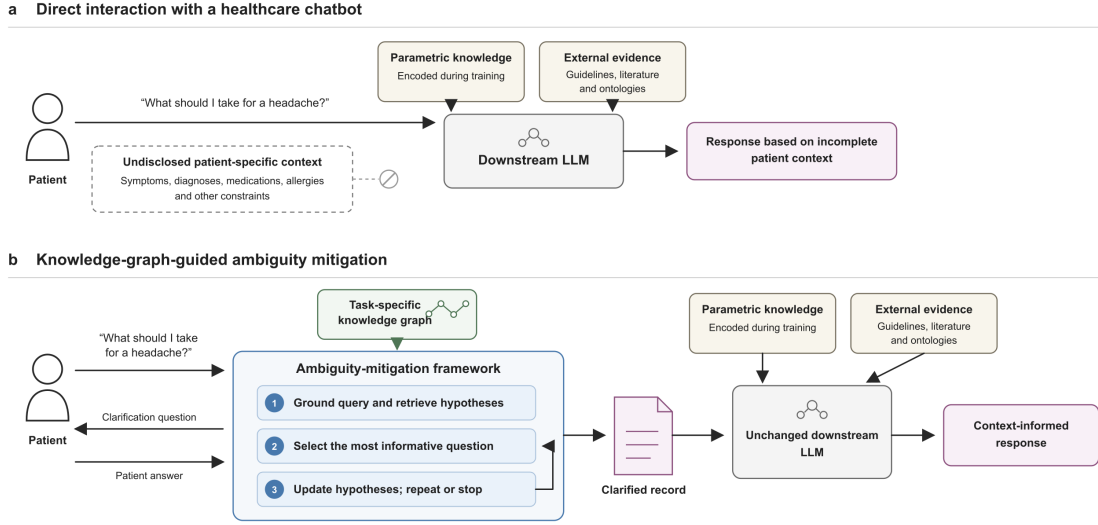


Figure 1: **Mitigating patient-context ambiguity before downstream language-model generation.** **a**, In direct interaction with a healthcare chatbot, the downstream large language model (LLM) can use knowledge encoded during training and, when available, external evidence. However, decision-relevant patient-specific context may remain unavailable because it has not been elicited. The response is therefore generated from incomplete patient context. **b**, The knowledge-graph-guided ambiguity-mitigation framework operates between the patient-facing interface and an otherwise unchanged downstream LLM. The framework grounds the query in a task-specific knowledge graph, retrieves graph-supported hypotheses and selects the most informative clarification question. The patient’s answer is used to update the active hypotheses, and the cycle repeats until a stopping condition is met. The resulting clarified record is then passed to the downstream LLM, which generates a response informed by the elicited context.

symptoms, diagnoses, medications, allergies, pregnancy status, and dietary restrictions. Although some of this information may be available through electronic health records or connected systems, much of it may be absent from a patient-facing chatbot interaction and accessible only by asking the patient.

Figure 1 contrasts these two interaction patterns. In a direct chatbot interaction, the downstream language model may have substantial parametric and external knowledge but still receive no information about the patient-specific factors that determine which response is appropriate. The proposed framework introduces an agentic layer between the patient and the downstream language model. This intermediary uses structured domain knowledge to identify unresolved hypotheses, asks targeted questions to obtain the missing context, and constructs a clarified prompt before final response generation.

Most existing approaches to improving healthcare language models focus on strengthening parametric or external knowledge. Medical instruction tuning and domain adaptation seek to improve the clinical knowledge encoded in model parameters, while retrieval-augmented generation and knowledge-graph grounding provide access to clinical evidence unavailable in the original prompt [10–12]. These approaches can reduce factual errors and knowledge staleness, but they cannot retrieve an undisclosed allergy, diagnosis, medication, symptom, or personal constraint from an external knowledge source.

Other approaches operate on the query or the model’s uncertainty. Rephrase-and-Respond first reformulates a question into a clearer form before generating an answer [13]. This can help when ambiguity is caused by wording, but rephrasing preserves the information content of the original query and cannot supply a missing patient variable. Uncertainty estimation and abstention can indicate that a model’s output may be

unreliable, but they do not necessarily identify which patient-specific question should be asked to resolve the underlying ambiguity [14, 15]. Thus, better phrasing, greater factual knowledge, and improved uncertainty estimation do not alone solve a problem whose decisive evidence has never been provided.

Interactive question asking offers a more direct mechanism for acquiring missing information. Prior work has investigated clarification in open-domain question answering, conversational search, and clinical reasoning [16–19]. However, directly prompting a language model to decide whether information is missing, generate a follow-up question, interpret the response, and determine when to stop leaves the complete clarification process dependent on the same model that initially received the underspecified query. In the MediQ clinical benchmark, prompting language models to ask questions did not consistently improve performance, demonstrating that useful information seeking does not arise reliably from prompting alone [19]. Knowledge-graph-based clarification provides a more explicit representation of candidate entities and discriminating relations [20], but much of this work addresses factual or entity ambiguity, where the missing information concerns an intended referent or external fact rather than patient-specific context that must be obtained from the user.

To address this gap, we introduce a knowledge-guided agentic framework that operates between the patient and an otherwise unchanged downstream language model. The framework interprets the initial query and uses a task-specific knowledge graph to construct a set of plausible hypotheses consistent with the available information. When multiple hypotheses remain, the agent identifies a patient-context variable that can distinguish among them and selects a targeted follow-up question. The patient’s answer updates the hypothesis space and guides subsequent questions. This process continues until the available information supports a specific answer, supports a clinically coherent group of answers, or remains insufficient for a supported response. The original query and the elicited context are then combined into a clarified prompt for the downstream language model. The framework therefore separates context acquisition from final natural-language generation: the agent determines what information is missing and acquires it, while the downstream model generates the final response from a better-specified input.

The framework is evaluated using two controlled benchmarks representing different forms of patient-context ambiguity. In the symptom–diagnosis task, complete synthetic patient records generated using Synthea establish reference diagnoses, after which clinically relevant symptoms are masked to create underspecified queries [21, 22]. A symptom–diagnosis knowledge graph derived from the Unified Medical Language System supports hypothesis construction and question selection [23]. In the dietary-safety task, the visible query identifies a food but omits the health condition required to determine whether that food is appropriate. In both tasks, responses to the agent’s follow-up questions are drawn from the hidden complete case, allowing the consequences of context acquisition to be evaluated against a known reference outcome. The same agent architecture is used across both domains, while the task-specific knowledge graph and output representation are changed.

This study makes three principal contributions. First, it formalizes patient-context ambiguity as a distinct problem in patient-facing healthcare chatbots, separating missing personal information from unclear wording, external knowledge gaps, and uncertain user intent. Second, it introduces a knowledge-guided agentic intermediary that actively elicits decision-relevant patient context and transforms an underspecified query into a clarified prompt without modifying or fine-tuning the downstream language model. Third, it evaluates the distinction between reformulating existing information and acquiring new information across two healthcare tasks and five downstream language models, assessing both answer performance and downstream predictive uncertainty under repeated generation.

2 Results

2.1 Evaluation overview

The evaluation included two health-related tasks with different output structures: symptom-based diagnosis retrieval and dietary safety classification. Both evaluation sets were constructed so that the intended answer could not be uniquely determined from the information initially presented to the model. The missing patient-specific information was retained separately and could be recovered only through the clarification process.

The diagnosis benchmark was derived from Synthea, an open-source synthetic patient-record generator designed to produce realistic longitudinal health records based on population-level epidemiological patterns [21]. For each eligible case, the reference diagnosis and the complete associated symptom set were retained from the original synthetic record without manually changing the diagnosis or rewriting the symptom content. Controlled ambiguity was then introduced by randomly withholding a non-empty subset of symptoms while retaining at least one symptom in the initial query. Cases were retained only when the remaining visible symptoms were compatible with multiple candidate diagnoses, ensuring that the initial query remained genuinely underspecified rather than simply containing fewer words. The complete symptom set was stored separately and was used by the simulated user to answer clarification questions. The evaluation cohort contained 1,034 cases spanning 32 diagnosis labels. Each complete case contained a mean of 8.22 symptoms, of which 4.16 were initially visible and 4.07 were withheld, corresponding to approximately 50.5% visible and 49.5% withheld diagnostic evidence. No patient contributed cases to more than one dataset partition.

The dietary-safety benchmark contained 487 binary queries spanning 38 patient health contexts. Each instance paired a food-related query with a patient-specific context and a reference OK or NOT OK decision. Starting from a fully specified query, the patient-context phrase was removed while the food request and reference decision were left unchanged. The removed context was stored separately and was available to the simulated user during clarification. This controlled masking procedure created queries for which the safety decision could not be resolved from the visible food description alone. The withheld information represented clinically relevant factors including metabolic conditions such as diabetes and hypertension, gastrointestinal disorders such as GERD and IBS, food allergies and intolerances, dietary restrictions, and pregnancy-related context. The final evaluation set contained 301 OK and 186 NOT OK queries.

The clinical relationships used during dietary clarification were represented in FoodSafetyKG. Condition concepts and their hierarchical relationships were normalized using the UMLS Metathesaurus [23]. The food side included 298 curated food expressions linked to ingredients and 60 nutritional or biochemical food properties. Risk and safety relationships between these properties and patient conditions were derived from 14 clinical practice guidelines covering areas including diabetes, cardiovascular disease, gastrointestinal disorders, kidney disease, pregnancy, food allergy, and other dietary-risk contexts. USDA FoodData Central was additionally used as a compositional food-data source; however, the benchmark evaluation used the curated food-expression layer rather than the complete USDA food-item layer. Complete knowledge-graph construction and guideline provenance are provided in the Methods and Supplementary Methods.

Three configurations were evaluated with five downstream language models: GPT-5.5, Claude Opus 4.8, Gemini 3.1 Pro, LLaMA 3.3 70B, and Mistral Large. In the BASIC configuration, each model answered the initially incomplete query directly. In Rephrase-and-Respond (RAR), the model first reformulated the query but received no additional patient-specific information. In the AGENT configuration, targeted follow-up questions were used to acquire the withheld patient context before the resulting information was provided to the downstream model.

Diagnosis retrieval was evaluated at broad diagnostic-category, ICD-grouping, and exact-diagnosis levels using Top-1 accuracy and Recall@5. Selective results were calculated among answered cases, whereas

Table 2: **Overview of the two ambiguity-mitigation evaluation sets.** Both evaluation sets were curated so that the intended output could not be uniquely determined from the information initially provided. Missing patient-specific information was subsequently made available through the clarification process.

	Symptom-based diagnosis retrieval	Dietary safety classification
Data source	Curated synthetic patient records	Curated food and health-condition resources
Evaluation instances	1,034	487
Output space	32 diagnosis labels	OK or NOT OK
Information initially provided	Partial symptom profile	Food-related question without complete health context
Information initially withheld	Approximately 49.5% of the symptoms recorded in the complete case	Patient-specific health conditions relevant to the queried food
Information initially visible	Approximately 50.5% of the symptoms recorded in the complete case	Queried food and the user’s stated request
Reference information used during clarification	Complete symptom profile	Complete patient health context
Health conditions represented	–	38
Label distribution	32 diagnosis classes	301 OK and 186 NOT OK
Primary evaluation measures	Category-level, ICD-level, and exact diagnosis retrieval	MCC, recall, precision, and classification error rates

overall results counted abstentions as incorrect. The Agent abstained on 35 of the 1,034 diagnosis cases, leaving 999 answered cases; the same cases were unanswered across all five downstream models. No abstentions occurred in the dietary-safety task.

Dietary safety classification was evaluated using Matthews correlation coefficient (MCC) as the primary summary measure, together with accuracy, recall, precision, F1 score, and false-negative and false-positive rates. Because the dataset was imbalanced, with 301 OK and 186 NOT OK queries, accuracy alone could mask poor performance on the less frequent class. MCC was therefore used as the primary measure because it incorporates all four confusion-matrix outcomes. The NOT OK label was treated as the positive class so that recall and the false-negative rate reflected the system’s ability to identify foods associated with a relevant health risk.

Table 2 summarizes the two evaluation sets.

2.2 Clarification improved diagnosis retrieval

Clarification substantially improved diagnosis retrieval across all five downstream language models, with the largest gains observed at the ICD and exact-diagnosis levels (Table 3). Baseline configurations often identified the correct broad diagnostic category, but their performance declined markedly when a more specific diagnosis was required.

For exact Top-1 retrieval, overall Agent accuracy ranged from 57.7% to 71.1%, compared with 0.6% to 11.0% for the BASIC and RAR configurations. The same pattern was observed for exact Recall@5. The Agent included the reference diagnosis among its five outputs in 90.6% to 90.9% of all cases, whereas exact Recall@5 remained between 2.0% and 16.2% for the baseline configurations.

Clarification also produced large improvements at the ICD level. Overall ICD Top-1 accuracy ranged from 78.8% to 80.6% for the Agent, compared with 27.2% to 45.9% for the baselines. Overall ICD Recall@5

ranged from 91.6% to 91.9% for the Agent, whereas the corresponding baseline values ranged from 56.3% to 67.5%. These results indicated that the additional symptom information improved both exact diagnosis selection and placement of the correct diagnosis among the leading candidates.

At the broad-category level, the difference was smaller. Overall Agent Top-1 accuracy ranged from 89.0% to 90.8%, compared with 74.7% to 83.8% for the baselines. However, category Recall@5 was already high without clarification, ranging from 92.9% to 96.8% across the baseline configurations. For GPT-5.5 and Gemini 3.1 Pro, some baseline configurations achieved higher category Recall@5 than the Agent. This pattern suggested that the initially visible symptoms were often sufficient to identify a broadly relevant diagnostic family, while the withheld information was more important for distinguishing among diagnoses within that family.

The exact-label distribution was concentrated, with viral sinusitis, acute viral pharyngitis, and acute bronchitis accounting for 67.8% of the evaluation set. Viral sinusitis alone occurred in 300 of the 1,034 cases, corresponding to an exact Top-1 accuracy of 29.01% for a constant predictor that returned this diagnosis for every query. Every BASIC and RAR configuration performed below this descriptive reference, whereas every Agent configuration performed above it. This comparison was used as a sanity check rather than as a clinically meaningful baseline.

The Agent abstained on 35 cases, representing 3.4% of the evaluation set. Because the abstention decision occurred before downstream language generation, the same cases were unanswered for all five models. Selective exact Top-1 accuracy ranged from 59.8% to 73.6%, compared with 57.7% to 71.1% when abstentions were counted as incorrect. The limited difference between selective and overall results showed that the observed improvement was not produced by excluding a large proportion of difficult cases.

Table 3: **Symptom-based diagnosis retrieval on the curated evaluation set** ($n = 1,034$). Performance was evaluated at three levels of specificity: broad diagnostic category, ICD-level grouping, and exact diagnosis name. Selective results include only cases for which the framework returned an answer. Overall results include all evaluation cases, with abstentions counted as incorrect. Because the BASIC and RAR configurations did not abstain, their selective and overall values are identical. Bold values indicate the best configuration within each downstream model and metric column. Underlined values indicate the highest value in the corresponding column across all models and configurations. Category and ICD values rely on reference mappings that were not independently clinician-adjudicated, whereas exact-name matching does not require these mappings. All values are percentages.

Model	Configuration	Abstain	Top-1, selective			Recall@5, selective			Top-1, overall			Recall@5, overall		
			Category	ICD	Exact	Category	ICD	Exact	Category	ICD	Exact	Category	ICD	Exact
GPT-5.5	Agent	3.4	92.1	82.2	64.3	96.9	95.1	94.1	89.0	79.4	62.1	93.6	91.9	90.9
GPT-5.5	RaR	0.0	80.3	27.5	1.7	95.6	60.5	2.2	80.3	27.5	1.7	95.6	60.5	2.2
GPT-5.5	Basic	0.0	80.9	27.2	1.6	95.6	60.1	2.0	80.9	27.2	1.6	95.6	60.1	2.0
Claude Opus 4.8	Agent	3.4	94.0	81.6	72.6	96.7	94.9	93.9	90.8	78.8	70.1	93.4	91.7	90.7
Claude Opus 4.8	RaR	0.0	79.2	33.3	4.5	94.8	63.4	10.9	79.2	33.3	4.5	94.8	63.4	10.9
Claude Opus 4.8	Basic	0.0	75.2	36.3	1.5	92.9	67.2	4.0	75.2	36.3	1.5	92.9	67.2	4.0
Gemini 3.1 Pro	Agent	3.4	92.1	82.3	59.8	96.9	95.1	94.1	89.0	79.5	57.7	93.6	91.9	90.9
Gemini 3.1 Pro	RaR	0.0	81.2	29.3	0.8	96.6	56.3	3.3	81.2	29.3	0.8	96.6	56.3	3.3
Gemini 3.1 Pro	Basic	0.0	83.8	28.6	0.6	96.8	67.5	5.7	83.8	28.6	0.6	96.8	67.5	5.7
LLaMA 3.3 70B	Agent	3.4	92.2	83.4	73.6	97.2	94.9	93.9	89.1	80.6	71.1	93.9	91.7	90.7
LLaMA 3.3 70B	RaR	0.0	81.7	44.5	10.0	94.9	62.7	15.7	81.7	44.5	10.0	94.9	62.7	15.7
LLaMA 3.3 70B	Basic	0.0	81.1	45.9	11.0	93.9	62.1	16.2	81.1	45.9	11.0	93.9	62.1	16.2
Mistral Large	Agent	3.4	93.0	82.1	71.2	97.5	94.8	93.8	89.8	79.3	68.8	94.2	91.6	90.6
Mistral Large	RaR	0.0	76.3	37.1	7.4	94.7	66.4	12.3	76.3	37.1	7.4	94.7	66.4	12.3
Mistral Large	Basic	0.0	74.7	38.9	2.7	93.7	66.3	5.1	74.7	38.9	2.7	93.7	66.3	5.1

Table 4: **Dietary safety classification on 487 queries.** The positive class is NOT OK. A false negative denotes an unsafe food classified as safe, whereas a false positive denotes an unnecessary restriction. MCC is reported because the dataset contains 301 OK and 186 NOT OK queries. Bold values indicate the best result within each downstream-model block and metric column; * marks the best configuration for each model, ranked by MCC. Higher values are better for accuracy, MCC, F1, recall, and precision, whereas lower values are better for false-negative rate, false-positive rate, and false-negative count. Accuracy, recall, precision, and error rates are percentages.

Model	Configuration	Accuracy	MCC	F1	Recall	Precision	FN rate	FP rate	FN (<i>n</i>)
GPT-5.5	Agent *	89.3	0.783	0.869	92.5	81.9	7.5	12.6	14
GPT-5.5	RaR	83.2	0.640	0.747	65.1	87.7	34.9	5.6	65
GPT-5.5	Basic	88.3	0.757	0.852	88.2	82.4	11.8	11.6	22
Claude Opus 4.8	Agent *	91.8	0.837	0.900	96.8	84.1	3.2	11.3	6
Claude Opus 4.8	RaR	83.4	0.650	0.741	62.4	91.3	37.6	3.7	70
Claude Opus 4.8	Basic	81.3	0.607	0.760	77.4	74.6	22.6	16.3	42
Gemini 3.1 Pro	Agent	90.3	0.795	0.872	86.0	88.4	14.0	7.0	26
Gemini 3.1 Pro	RaR	88.3	0.751	0.846	83.9	85.2	16.1	9.0	30
Gemini 3.1 Pro	Basic *	89.5	0.799	0.877	97.3	79.7	2.7	15.3	5
LLaMA 3.3 70B	Agent *	86.9	0.750	0.848	95.7	76.1	4.3	18.6	8
LLaMA 3.3 70B	RaR	85.6	0.726	0.834	94.6	74.6	5.4	19.9	10
LLaMA 3.3 70B	Basic	83.8	0.701	0.818	95.7	71.5	4.3	23.6	8
Mistral Large	Agent *	86.4	0.741	0.843	95.2	75.6	4.8	18.9	9
Mistral Large	RaR	84.2	0.718	0.826	98.4	71.2	1.6	24.6	3
Mistral Large	Basic	85.0	0.718	0.829	95.2	73.4	4.8	21.3	9

2.3 Clarification improved dietary-safety classification

Clarification generally improved dietary-safety classification across the five downstream language models (Table 4). The Agent achieved the highest MCC for GPT-5.5, Claude Opus 4.8, LLaMA 3.3 70B, and Mistral Large. For Gemini 3.1 Pro, the BASIC configuration had the highest MCC, although the Agent achieved the highest accuracy and the lowest false-positive rate.

The effect of clarification was most apparent in the balance between identifying unsafe foods and avoiding unnecessary restrictions. For GPT-5.5 and Claude Opus 4.8, the Agent substantially reduced false-negative errors relative to both baseline configurations while maintaining comparatively strong precision. For LLaMA 3.3 70B, Agent and BASIC had the same recall of 95.7% and the same false-negative count of eight, but the Agent reduced the false-positive rate from 23.6% to 18.6%. This increased precision from 71.5% to 76.1% and raised MCC from 0.701 to 0.750.

A similar pattern was observed for Mistral Large. Agent and BASIC had the same recall of 95.2%, but the Agent produced fewer false positives and higher precision, resulting in a higher MCC. The RAR configuration achieved the highest recall for this model, but this gain was accompanied by a false-positive rate of 24.6%, showing that greater sensitivity alone did not provide the best overall classification balance.

Gemini 3.1 Pro showed a different trade-off. The BASIC configuration achieved the highest recall and MCC, whereas the Agent produced the highest accuracy, precision, and lowest false-positive rate. This result indicated that clarification did not dominate every metric for every model, but generally improved the balance between detecting unsafe foods and avoiding unnecessary restrictions.

Rephrase-and-Respond did not provide a consistent advantage over direct prompting. Its effect varied across models and often involved a trade-off between recall and false-positive error. Overall, the Agent produced the best MCC for four of the five downstream models, while the Gemini results showed that the benefit of clarification could depend on the model and the relative importance assigned to false negatives and false positives.

2.4 Clarification produced more consistent performance across the evaluated models

This analysis examined whether the performance of each approach changed substantially when a different downstream language model was used. The term *configuration* refers to one of the three evaluated approaches: BASIC, in which the model answered the incomplete query directly; RAR, in which the incomplete query was first rephrased and then answered; and AGENT, in which missing patient-specific information was obtained through clarification before the downstream model generated the answer.

The analysis did not measure growth over time or improvement within an individual model. Instead, for each configuration and evaluation metric, the corresponding scores from GPT-5.5, Claude Opus 4.8, Gemini 3.1 Pro, LLaMA 3.3 70B, and Mistral Large were collected and summarized. This calculation was performed separately for BASIC, RAR, and AGENT. The mean represented the average performance across the five models, while the population standard deviation summarized how widely the five model-specific scores were distributed around that mean.

In Figure 2, each marker denotes the mean across the five downstream models, and each horizontal error bar extends one population standard deviation on either side of the mean. A shorter error bar therefore indicates that the five models produced more similar results under that configuration. Between-model variation was calculated across the five models and could not be calculated for one model alone.

Variation was interpreted together with mean performance. A small standard deviation was considered favorable only when the corresponding mean was also strong, because five models could show little variation by all performing similarly poorly. The arrows in Figure 2 indicate whether higher or lower values represent better performance for each metric.

For overall category Top-1 diagnosis accuracy, the Agent achieved a mean of 89.5% across the five models, with a population standard deviation of 0.72 percentage points. The corresponding means were 79.7% for RAR and 79.1% for BASIC, with standard deviations of 1.93 and 3.59 percentage points, respectively (Figure 2). Category-level accuracy was therefore both higher on average and less dependent on the selected downstream model after clarification.

The difference was more pronounced at the ICD level. Overall ICD Top-1 accuracy under the Agent had a mean of 79.5% and a population standard deviation of 0.57 percentage points. The corresponding means were 34.3% for RAR and 35.4% for BASIC, with standard deviations of 6.08 and 6.89 percentage points. Thus, ICD-level performance varied substantially across downstream models when the incomplete query was answered directly or rephrased, whereas performance after clarification remained similar across the model panel.

Exact Recall@5 showed the clearest diagnosis pattern. The Agent achieved a mean of 90.8%, with a population standard deviation of only 0.12 percentage points. In comparison, RAR achieved a mean of 8.9% with a standard deviation of 5.24 percentage points, while BASIC achieved a mean of 6.6% with a standard deviation of 4.98 percentage points. Clarification therefore combined substantially higher exact retrieval performance with markedly smaller differences among the five downstream models.

A similar pattern was observed for dietary safety. MCC was calculated separately for each downstream model under each configuration and then summarized across the five model-specific values. The Agent achieved the highest mean MCC, at 0.781, and the lowest population standard deviation, at 0.034. The corresponding means were 0.697 for RAR and 0.716 for BASIC, with standard deviations of 0.044 and 0.064, respectively. The balance between identifying unsafe foods and avoiding unnecessary restrictions therefore varied less across models after clarification.

The dietary false-negative rate showed the same pattern in the favorable lower direction. The Agent had the lowest mean false-negative rate, at 6.8%, and the lowest population standard deviation, at 3.87 percentage

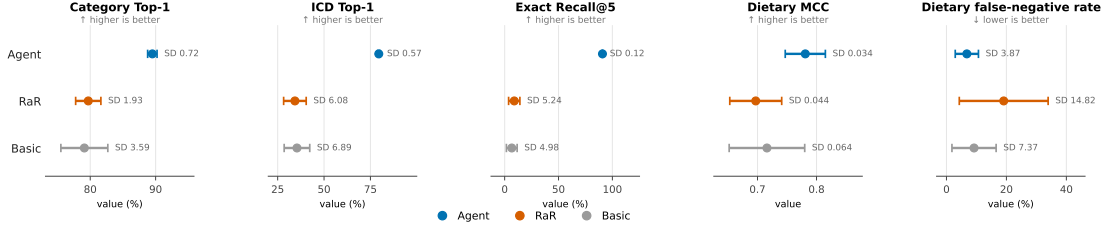


Figure 2: **Between-model performance and variation under each configuration.** Markers denote the mean across the five evaluated downstream language models, and horizontal error bars denote ± 1 population standard deviation across the five model-specific values. Diagnosis metrics were computed over all 1,034 cases, with abstentions counted as incorrect, and dietary metrics were computed over all 487 queries. Higher values indicate better performance for category Top-1 accuracy, ICD Top-1 accuracy, exact Recall@5, and dietary MCC, whereas a lower value indicates better performance for the dietary false-negative rate, as shown by the arrows above each panel. For every displayed metric, the Agent attained both a better mean and a smaller between-model standard deviation than either baseline, indicating stronger average performance and less dependence on the selected downstream model. MCC is unitless; all other quantities are expressed in percentage points. Exact Top-1 diagnosis accuracy was omitted because its near-zero baseline means made dispersion alone misleading.

points. The corresponding means were 19.1% for RAR and 9.2% for BASIC, with standard deviations of 14.82 and 7.37 percentage points. Both the average frequency of missed unsafe foods and its dependence on the downstream model were therefore lower under the Agent configuration.

Because the Agent abstained on 35 diagnosis cases, the diagnosis dispersion analysis was also repeated using the same 999 answered cases for all three configurations. The Agent retained the lowest variation for the representative category, ICD, and Recall@5 measures. Its greater cross-model consistency was therefore not explained by evaluating a different or smaller set of cases.

Exact Top-1 diagnosis accuracy was the main exception and was omitted from Figure 2. Its between-model standard deviation was higher under the Agent than under the baseline configurations. However, mean exact Top-1 accuracy was 66.0% for the Agent, compared with only 3.5% for BASIC and 4.9% for RAR. The lower baseline variation therefore occurred because all five baseline models performed near zero, rather than because the baselines provided better or more reliable exact diagnosis retrieval.

Overall, clarification produced stronger mean performance and lower between-model variation for every metric shown in Figure 2. Performance under the Agent was therefore less dependent on the selected downstream language model than performance under either baseline. This analysis was descriptive and applied only to the five models included in the evaluation.

2.5 Clarification reduced uncertainty in diagnosis outputs

This exploratory analysis examined how much the generated diagnosis varied when the same query was repeated. Two diagnosis queries were selected, and each model–configuration pair was evaluated over 500 repetitions per query. The purpose was not to measure diagnostic accuracy again, but to characterize the dispersion of the model’s own repeated outputs.

Three complementary quantities were reported in Table 5. K denoted the number of distinct exact diagnosis strings returned across the 500 repetitions. H_{exact} measured predictive entropy over those exact strings, with lower values indicating that the same wording was returned more consistently. H_{cat} measured entropy after the diagnosis strings were mapped to clinical categories. This second measure separated variation in wording from variation in the underlying clinical conclusion. A model could therefore have nonzero

exact-string entropy while retaining zero category-level entropy if it used different expressions for the same clinical category.

The repeated-sampling protocol differed between the Agent and the two baseline configurations. For the Agent, the clarification trajectory, terminal state, and final hypothesis set were generated once and then held fixed across the 500 repetitions. The repeated runs therefore sampled only the downstream generation of an already clarified decision. For BASIC and RAR, the complete response was generated again on every repetition. The values in Table 5 should therefore be interpreted as observed output dispersion under these sampling protocols, rather than as a fully symmetric measure of end-to-end uncertainty.

Within this protocol, the Agent produced substantially less category-level variation than either baseline. Across the ten Agent item–model pairs, mean exact-string entropy was 0.170 bits and mean category-level entropy was 0.017 bits. The corresponding mean category-level entropies were 0.320 bits for BASIC and 0.338 bits for RAR. Thus, the remaining Agent variation occurred primarily in the exact wording of the diagnosis rather than in the resolved clinical category.

This distinction was particularly clear for the ischemic heart disease query. Under the Agent configuration, Mistral Large returned three distinct diagnosis strings, resulting in an exact-string entropy of 1.145 bits, while all outputs mapped to the same clinical category and produced a category-level entropy of 0 bits. Claude Opus 4.8 showed a similar pattern, with an exact-string entropy of 0.335 bits and a category-level entropy of 0 bits. These examples showed that variation in surface wording did not necessarily indicate variation in the clinical conclusion.

The baseline configurations showed greater dispersion on the same query. They returned as many as 17 distinct diagnosis strings, and category-level entropy reached 1.128 bits. For example, Gemini 3.1 Pro under RAR returned 17 distinct diagnoses, with an exact-string entropy of 2.703 bits and a category-level entropy of 0.930 bits. This indicated variation not only in wording but also in the clinical category assigned across repetitions.

Entropy characterized consistency rather than correctness. A low entropy value meant that a model repeatedly produced the same answer or category, but it did not establish that the answer agreed with the reference diagnosis. For the acute viral pharyngitis query, several baseline configurations repeatedly returned the same alternative diagnosis and therefore had zero entropy despite being consistently incorrect. Accuracy was evaluated separately in the main diagnosis analysis and was not used in calculating K , H_{exact} , or H_{cat} .

Overall, the Agent showed lower output dispersion at the clinical-category level across the two evaluated queries, while the baselines exhibited both lexical and clinical-category variation. Because only two queries were examined and the Agent decision was fixed before repeated generation, these findings were treated as an exploratory analysis of downstream output stability rather than an estimate of uncertainty across the full diagnosis evaluation set.

The two probe queries also revealed that low entropy alone did not imply correctness. For the acute viral pharyngitis query, several baseline configurations returned acute tonsillitis on all 500 repetitions. These configurations had zero entropy but zero accuracy because the same incorrect diagnosis was returned consistently. Other baseline configurations showed the opposite pattern, producing a broad distribution of diagnoses with high entropy. For example, Gemini 3.1 Pro under RAR returned 17 distinct diagnoses for the ischemic heart disease query, with an exact-string entropy of 2.703 bits.

These findings showed that the baseline configurations exhibited both consistent error and variable error, whereas the Agent produced the correct modal diagnosis in every evaluated item–model pair and showed minimal variation at the clinical-category level. Because only two queries were examined and the Agent trajectory was held fixed, these results were treated as an exploratory analysis of downstream output stability rather than an estimate of uncertainty across the full evaluation set.

Table 5: **Diagnosis-output uncertainty under repeated sampling.** Two randomly selected diagnosis queries were evaluated using 500 independent repetitions for each model and configuration. K denotes the number of distinct exact diagnosis strings returned, H_{exact} denotes predictive entropy over exact diagnosis strings, and H_{cat} denotes entropy after mapping outputs to their clinical categories. Item 0 had the reference diagnosis acute viral pharyngitis, and Item 98 had the reference diagnosis ischemic heart disease. In the Agent condition, the symbolic clarification trajectory and terminal hypothesis were held fixed, so the repetitions resampled only downstream language generation. Outputs that could not be mapped to a clinical category were retained as distinct categories rather than excluded. Bold values identify the lowest-uncertainty configuration within each model block, and $\star$ marks that configuration. Lower values indicate less variation across repeated outputs. Entropy measures the dispersion of the generated answers and does not, by itself, indicate whether the resulting diagnosis was correct. Entropy is reported in bits.

Model	Configuration	Item 0: acute viral pharyngitis			Item 98: ischemic heart disease		
		K	H_{exact}	H_{cat}	K	H_{exact}	H_{cat}
GPT-5.5	Agent $\star$	1	0.000	0.000	1	0.000	0.000
GPT-5.5	RaR	1	0.000	0.000	6	1.006	0.988
GPT-5.5	Basic	1	0.000	0.000	5	1.232	1.128
Claude Opus 4.8	Agent $\star$	1	0.000	0.000	2	0.335	0.000
Claude Opus 4.8	RaR	11	2.092	0.000	9	2.044	0.511
Claude Opus 4.8	Basic	1	0.000	0.000	16	1.161	0.932
Gemini 3.1 Pro	Agent $\star$	1	0.000	0.000	2	0.094	0.094
Gemini 3.1 Pro	RaR	3	0.058	0.000	17	2.703	0.930
Gemini 3.1 Pro	Basic	2	0.053	0.000	15	1.968	1.056
LLaMA 3.3 70B	Agent $\star$	3	0.074	0.021	2	0.053	0.053
LLaMA 3.3 70B	RaR	6	1.897	0.000	7	1.208	0.205
LLaMA 3.3 70B	Basic	5	1.530	0.000	5	0.847	0.021
Mistral Large	Agent $\star$	1	0.000	0.000	3	1.145	0.000
Mistral Large	RaR	3	0.236	0.000	4	1.411	0.741
Mistral Large	Basic	1	0.000	0.000	3	0.344	0.067

2.6 Clarification reduced uncertainty in dietary-safety decisions

This exploratory analysis examined how consistently each approach classified the same dietary query across repeated generations. Two queries were selected, one labeled NOT OK and one labeled OK. Each model–configuration pair was evaluated over 500 repetitions for each query.

Uncertainty was calculated separately for the two queries. Predictive entropy for the NOT OK-labeled query was reported as H_{unsafe} , and entropy for the OK-labeled query was reported as H_{safe} . Mean entropy was calculated as the arithmetic mean of these two query-specific values. An entropy of 0 bits indicated that the same label was returned in all 500 repetitions of that query, whereas a value approaching 1 bit indicated a more even division between OK and NOT OK.

The 500 outputs from each query were pooled only for the accompanying classification measures, producing 1,000 decisions per model–configuration pair. Because the two queries represented opposite reference classes, the pooled outputs formed a balanced binary set of 500 positive and 500 negative observations. This allowed accuracy, MCC, recall, precision, and false-positive rate to be calculated. These classification measures described whether the repeated decisions were correct, whereas the two entropy values described how much the decisions varied within each query.

As in the diagnosis analysis, the repeated-sampling protocol differed between the Agent and the two baseline configurations. For the Agent, the confirmed patient context and final dietary-safety decision were generated once and then held fixed across repetitions. The repeated runs therefore measured variation in the downstream expression of an already clarified decision. For BASIC and RAR, the complete response was regenerated on every repetition. The results in Table 6 should therefore be interpreted as observed output dispersion under these sampling protocols, rather than as a fully symmetric comparison of end-to-end uncertainty.

Under this protocol, all five Agent configurations returned the same decision on every repetition for both queries, resulting in $H_{\text{unsafe}} = 0$, $H_{\text{safe}} = 0$, and a mean entropy of 0 bits. The Agent also classified both queries correctly in every repetition. The zero-entropy result therefore reflected stable downstream expression of the fixed clarified decision.

Variation in the baseline configurations was concentrated mainly in the OK-labeled query. For the NOT OK-labeled query, entropy did not exceed 0.343 bits, and 11 of the 15 model–configuration rows had zero entropy. In contrast, entropy for the safe query reached 0.999 bits for Mistral Large under BASIC, indicating an almost even division between OK and NOT OK across the 500 repetitions. Gemini 3.1 Pro under BASIC also showed substantial variation on the safe query, with an entropy of 0.689 bits.

The repeated decisions also showed that low entropy did not necessarily indicate correct classification. LLaMA 3.3 70B under BASIC produced zero entropy for both queries because it returned NOT OK on all 1,000 repetitions. This behavior correctly classified the unsafe query but consistently misclassified the safe query, resulting in 50.0% accuracy and an MCC of 0. Entropy therefore measured consistency of the repeated outputs, not agreement with the reference labels.

Rephrase-and-Respond did not produce a consistent reduction in output dispersion. Mean entropy across the two queries and five models was 0.169 bits for BASIC and 0.146 bits for RAR. Although RAR reduced entropy for some models, substantial variation remained for others. For example, Mistral Large under RAR had a safe-query entropy of 0.869 bits and a mean entropy of 0.434 bits.

Overall, the Agent produced fully consistent dietary-safety decisions under the fixed-decision protocol, whereas the baselines showed either variable decisions or highly consistent but incorrect decisions. Because only two queries were examined and the Agent decision was fixed before repeated generation, these findings were treated as an exploratory analysis of downstream output stability rather than an estimate of uncertainty across the full dietary evaluation set.

Table 6: **Dietary-safety uncertainty under repeated sampling.** One NOT OK-labeled query and one OK-labeled query were each evaluated using 500 independent repetitions per model and configuration, producing 1,000 binary decisions per row. The positive class was NOT OK. H_{unsafe} and H_{safe} denote predictive entropy for the NOT OK- and OK-labeled queries, respectively. Zero bits indicates a fully consistent output, while one bit indicates an approximately even split between the two labels. In the Agent condition, the confirmed patient context and final safety decision were held fixed across repetitions, so the repetitions resampled only downstream language generation. Bold values indicate the best result within each model block and metric column. The $\star$ symbol marks the best-performing configuration for each model, ranked first by MCC, then by accuracy and mean entropy. Exact ties are marked for both configurations. Low entropy indicates consistency, not necessarily correctness. Accuracy, recall, precision, and false-positive rate are percentages; MCC is unitless.

Model	Configuration	Classification over 1,000 repetitions					Predictive entropy (bits)		
		Accuracy	MCC	Recall	Precision	FP rate	H_{unsafe}	H_{safe}	Mean H
GPT-5.5	Agent $\star$	100.0	1.000	100.0	100.0	0.0	0.000	0.000	0.000
GPT-5.5	RaR	96.8	0.938	93.6	100.0	0.0	0.343	0.000	0.172
GPT-5.5	Basic $\star$	100.0	1.000	100.0	100.0	0.0	0.000	0.000	0.000
Claude Opus 4.8	Agent $\star$	100.0	1.000	100.0	100.0	0.0	0.000	0.000	0.000
Claude Opus 4.8	RaR	99.2	0.984	98.4	100.0	0.0	0.118	0.000	0.059
Claude Opus 4.8	Basic $\star$	100.0	1.000	100.0	100.0	0.0	0.000	0.000	0.000
Gemini 3.1 Pro	Agent $\star$	100.0	1.000	100.0	100.0	0.0	0.000	0.000	0.000
Gemini 3.1 Pro	RaR	99.8	0.996	99.6	100.0	0.0	0.038	0.000	0.019
Gemini 3.1 Pro	Basic	59.2	0.318	100.0	55.1	81.6	0.000	0.689	0.344
LLaMA 3.3 70B	Agent $\star$	100.0	1.000	100.0	100.0	0.0	0.000	0.000	0.000
LLaMA 3.3 70B	RaR	50.6	0.078	100.0	50.3	98.8	0.000	0.094	0.047
LLaMA 3.3 70B	Basic	50.0	0.000	100.0	50.0	100.0	0.000	0.000	0.000
Mistral Large	Agent $\star$	100.0	1.000	100.0	100.0	0.0	0.000	0.000	0.000
Mistral Large	RaR	64.5	0.412	100.0	58.5	71.0	0.000	0.869	0.434
Mistral Large	Basic	76.1	0.594	100.0	67.7	47.8	0.000	0.999	0.499

3 Discussion

Health-related questions can be clearly phrased while remaining clinically underspecified. A user may describe symptoms or ask whether a food is appropriate without mentioning the patient-specific information that determines which answer is most appropriate. The findings of this study support treating this missing personal context as a distinct source of ambiguity. Rather than requiring the downstream language model to infer an unobserved patient variable, the proposed framework attempted to identify and acquire that information before generation.

The diagnosis results illustrate why this distinction matters. Without clarification, the evaluated language models were often able to identify a broadly relevant diagnostic category, but their performance decreased substantially when the task required the correct ICD grouping or exact diagnosis. This pattern suggests that general medical knowledge was not always the primary limitation. The initially visible symptoms could often support a broad set of clinically related possibilities, while the withheld symptoms were needed to distinguish among diagnoses within that set. Clarification therefore changed the information available for the decision rather than simply changing how the original information was phrased.

This interpretation is also consistent with the limited effect of Rephrase-and-Respond. Reformulating an underspecified query may improve its wording or organization, but it cannot introduce a patient-specific fact that was never provided. Prior work on ambiguity in question answering has examined reformulation, alternative interpretations, clarification, and uncertainty estimation [3, 8, 13, 24–26]. These approaches address important forms of ambiguity, but latent user ambiguity poses a different information problem: the decisive variable may exist only with the user. In that setting, interaction is required to obtain new evidence rather than to reinterpret the evidence already present in the query.

The dietary-safety results provide a complementary example. Whether a food is appropriate can depend on a health condition that is not evident from the food request itself. Clarification generally improved the balance between identifying potentially unsafe foods and avoiding unnecessary restrictions, as reflected by the MCC results across the evaluated models. The improvement was not uniform across every metric or every model. In particular, Gemini 3.1 Pro achieved a slightly higher MCC under direct prompting than under the Agent configuration, although the Agent achieved higher accuracy and fewer false positives. This exception is important because it shows that acquiring additional context did not guarantee superiority on every evaluation measure. Its effect depended partly on the downstream model and on the relative consequences assigned to false-negative and false-positive decisions.

A second finding was that clarification reduced the dependence of performance on the particular downstream language model. For each metric shown in Figure 2, the Agent combined stronger mean performance with lower between-model variation than either baseline. The effect was particularly pronounced for the more specific diagnosis measures. One interpretation is that supplying more decision-relevant patient context reduces the amount of ambiguity that the downstream model itself must resolve. When the input remains underspecified, different models may rely on different assumptions or priors and consequently produce more variable outcomes. Providing the missing context constrains the downstream problem and may therefore make the final result less sensitive to model choice. This finding should not be interpreted as evidence that the framework is model-independent in a general sense, because only five downstream models were evaluated.

The repeated-sampling experiments provide additional, although exploratory, evidence for this interpretation. In the two diagnosis probes, Agent outputs showed very little variation at the clinical category level, even when different surface forms of the diagnosis were generated. The dietary probes similarly produced stable binary decisions after clarification. These results are consistent with the idea that removing uncertainty about patient context can reduce variability in downstream generation. However, predictive entropy

measures consistency, not correctness. A model can repeatedly produce the same incorrect answer and therefore have low entropy, as observed in several baseline conditions. This distinction is important when interpreting uncertainty measures for health-facing language models [27–29].

The uncertainty results also require a second qualification. In the Agent condition, the clarification trajectory and terminal decision were generated once and held fixed during repeated sampling, whereas the baseline configurations regenerated their complete response on each repetition. Agent entropy therefore measured the stability of downstream generation after a clarified state had already been reached. It did not measure end-to-end variability of the full clarification process. Consequently, the uncertainty experiments should be viewed as exploratory evidence about downstream output stability rather than as a direct comparison of total uncertainty across the complete systems.

The use of structured domain knowledge provided a practical mechanism for deciding what information to request. In both tasks, the knowledge graph defined relationships between observable information, possible hypotheses, and clinically relevant categories. This allowed question selection and stopping decisions to be separated from downstream language generation. Such separation may be useful in health-facing systems because it makes the source of a clarification request more explicit than allowing the language model to invent both the missing hypothesis and the question needed to test it. At the same time, the framework inherits the limitations of its knowledge representation. Missing, incomplete, or incorrect relations can prevent the appropriate hypothesis from being retrieved or can limit which clarification questions are available. Improving knowledge-graph coverage and provenance is therefore an important direction for future work.

A further consideration concerns which kinds of patient context the framework can currently acquire. Because context is obtained by asking, the accessible information is bounded by what the patient is aware of and able to report. Relatively stable attributes such as diagnoses, medications, allergies, pregnancy status, and dietary restrictions are usually known and can be stated directly, and both evaluated tasks relied on context of this kind together with symptoms the patient could perceive and describe. Other decision-relevant context is dynamic and frequently outside the patient’s awareness. Hydration status, sleep, physical activity, recent nutritional intake, and environmental exposures may all affect which response is appropriate, yet a patient asked about them directly may be unable to answer accurately. A third category is longitudinal: the accumulated record of an individual’s health events and behaviours over time, as represented in personal chronicle systems and electronic health records, which cannot be reconstructed within a single conversation. The present architecture separates the decision about which variable is missing from the mechanism used to obtain it. The controller selects a graph-defined variable, and the inner language model renders it as a question only because the patient is the assumed source. The same selection procedure could route a selected variable to a wearable data stream, an environmental data source, or a structured clinical record, and could query the patient only when no other source is available. We are beginning to examine this extension for dynamic context and regard longitudinal context as a further direction; neither is evaluated here, and the present results should therefore be read as covering the patient context that individuals can report themselves.

Several limitations constrain the conclusions that can be drawn from this study. First, the diagnosis benchmark was derived from Synthea rather than from real clinical encounters. Synthetic records support controlled experimentation but do not reproduce the full complexity of comorbidities, documentation variability, incomplete histories, and patient language encountered in practice [21]. Evaluation on real-world clinical data is needed to determine whether the observed effects persist under those conditions.

Second, the dietary-safety benchmark covered 487 queries across 38 patient contexts and therefore represented only a subset of the conditions, foods, medication interactions, and competing dietary considerations that may arise in practice. The benchmark relied on a curated food and condition representation supported by UMLS and clinical guidelines, but broader clinical review and evaluation across additional conditions

and dietary contexts would be necessary before drawing conclusions about real-world safety.

Third, clarification answers were supplied by simulation oracles using the complete underlying case information. Real users may misunderstand a question, provide uncertain or incomplete answers, use nonstandard terminology, or disclose several interacting conditions simultaneously. The current evaluation therefore measured performance when the requested context could be recovered reliably. Future studies should examine robustness to noisy responses, contradictory information, multiple comorbidities, and naturally occurring patient dialogue.

Fourth, the diagnosis category and ICD analyses depended on prespecified reference mappings rather than independently adjudicated clinical coding. Exact diagnosis-name matching did not require this mapping, but the higher-level results should therefore be interpreted as agreement with the study’s reference representation rather than as independently verified clinical coding accuracy.

Fifth, the uncertainty analysis included only two randomly selected queries from each task. Repeating each query 500 times provided a detailed estimate of output variation for those individual probes but did not estimate uncertainty across the full evaluation distribution. A larger query-level uncertainty study, ideally including repeated execution of the complete clarification trajectory, would be required to determine whether the observed stability generalizes across cases.

Finally, the study evaluated five downstream language models and two health-related tasks. The consistency of the main pattern across those models and tasks supports further investigation, but it does not establish generalization to other models, clinical domains, populations, or deployment settings. The framework was evaluated as an ambiguity-mitigation mechanism, not as an autonomous diagnostic or clinical decision system.

Taken together, the results suggest that some failures of health-facing language models may arise not because the model lacks relevant general medical knowledge, but because the question does not yet contain the patient-specific information needed to select among several plausible responses. In such cases, improving the final generation alone may be insufficient. A complementary strategy is to treat clarification as an information-acquisition step before generation: identify which patient-specific variable is missing, obtain it when possible, and avoid forcing specificity when the available evidence remains insufficient. The present findings provide controlled evidence for this approach across two health-related tasks and motivate evaluation with real users, clinician-reviewed data, and naturally occurring clinical ambiguity.

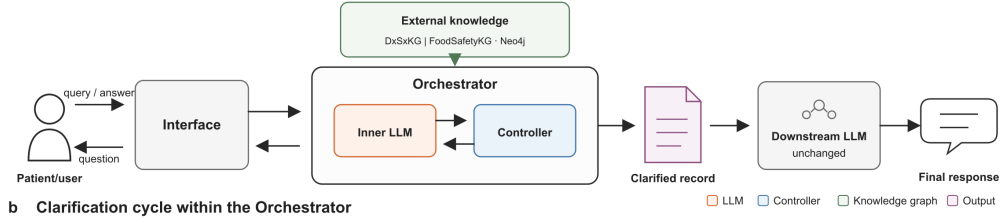
Methods

We developed a knowledge-graph-guided agentic framework to recover decision-relevant information missing from a user query before the query is passed to a downstream large language model (LLM). The framework maps the information already stated in the query to a set of hypotheses supported by a task-specific knowledge graph. It then identifies the unresolved fact that best distinguishes among those hypotheses and asks a targeted clarification question. The controller uses the answer to remove incompatible hypotheses and select the next question, if one remains. This process continues until the relevant context is resolved or the graph cannot support a further informative question. The downstream LLM therefore receives a clarified query and traceable graph evidence, rather than being required to infer unstated information.

The knowledge graph does not generate the final response. It defines which hypotheses are supported, which missing facts can distinguish them and how each possible answer changes the active hypothesis set. Domain knowledge is thus encoded in the graph, whereas clarification follows the same controller logic across tasks.

We implemented the framework in openCHA [30]. The implementation comprised three components:

a Framework placement



b Clarification cycle within the Orchestrator

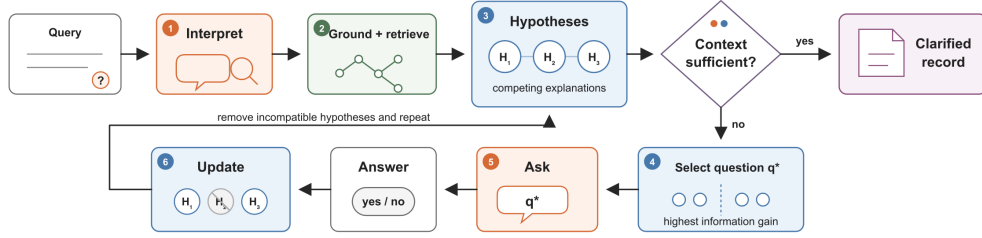


Figure 3: **Knowledge-graph-guided ambiguity-mitigation framework.** **a**, The framework operates between the user-facing Interface and an unchanged downstream large language model (LLM). The Interface exchanges the initial query, clarification questions and user answers. The Orchestrator combines an inner LLM with a deterministic controller and accesses a task-specific knowledge graph implemented in Neo4j. After clarification, the Orchestrator passes the clarified query and retained graph evidence to the downstream LLM for response generation. **b**, The inner LLM interprets the query, and graph grounding retrieves the supported candidate hypotheses. When the available context is insufficient, the controller selects the eligible question with the greatest expected information gain. The inner LLM expresses the selected variable as a user-facing question, and the controller uses the answer to remove incompatible hypotheses. The cycle repeats until the remaining evidence supports a task output, no informative question remains or the interaction limit is reached. Orange denotes inner-LLM operations, blue denotes deterministic-controller operations, green denotes knowledge-graph operations and purple denotes the clarified output.

an Interface that managed information exchange, an Orchestrator that executed the clarification workflow and an external knowledge source that constrained the hypotheses, questions and evidence (Fig. 3).

Interface. The Interface passed the user query and available conversation history to the Orchestrator, presented each clarification question and returned the answer. When clarification ended, it transferred the clarified query and evidence record to the downstream LLM, which generated the response outside the agent. The interactive implementation used Gradio; benchmark runs bypassed the graphical interface and supplied oracle answers programmatically.

Orchestrator. The Orchestrator coordinated a fixed inner model, GPT-5.5 accessed through the OpenAI API, and a deterministic clarification controller. When a query arrived, the inner model identified text that required graph grounding and proposed task-relevant search terms. The controller matched those terms to graph entities, discarded unmatched terms and retrieved the hypotheses supported by the matched entities. The inner model then made one expansion decision: whether an important interpretation of the visible query was absent from the retrieved hypotheses. If so, it proposed additional search terms, which affected the hypothesis set only when they resolved to eligible graph entities. The resulting hypotheses were then fixed as the starting point for clarification.

During clarification, the controller compared the unasked facts connected to the active hypotheses and selected the fact whose answer was expected to reduce uncertainty most. The inner model expressed that fact as a patient-facing question, and the Interface returned the answer. The controller then removed hypotheses that conflicted with the answer and repeated the cycle when another informative question remained.

When clarification ended, the controller fixed the question–answer history, remaining hypotheses, resolution state, stopping reason and supporting graph evidence. The inner model formatted this information as a structured clarified query for the downstream LLM; it could not change the final controller state.

Question selection and hypothesis updating were implemented deterministically as follows. Let $\mathcal{H}_0 = \{h_1, \dots, h_K\}$ denote the graph-supported hypotheses retrieved for a query. The controller assigned each hypothesis equal initial weight,

$$b_0(h) = \frac{1}{|\mathcal{H}_0|}, \quad h \in \mathcal{H}_0. \quad (1)$$

These weights were used to select questions and were not interpreted as calibrated clinical probabilities. If no graph-supported hypothesis was retrieved, the controller entered the unresolved state defined for that task.

At turn t , let $\mathcal{H}_t$ be the active hypotheses and $\mathcal{V}_t$ the eligible, previously unasked binary clarification variables. For variable v and answer $a \in \{0, 1\}$, $\mathcal{H}_t(v, a) \subseteq \mathcal{H}_t$ denotes the hypotheses compatible with that answer. Uncertainty in the normalized weight distribution b_t was measured by Shannon entropy,

$$H(b_t) = - \sum_{h \in \mathcal{H}_t} b_t(h) \log_2 b_t(h). \quad (2)$$

The controller selected the variable with the greatest expected reduction in entropy,

$$v_t^* = \arg \max_{v \in \mathcal{V}_t} \left\{ H(b_t) - \sum_{a \in \{0, 1\}} P(a \mid v, b_t) H(b_t \mid a, v) \right\}, \quad (3)$$

where $P(a \mid v, b_t) = \sum_{h \in \mathcal{H}_t(v, a)} b_t(h)$, and $H(b_t \mid a, v)$ is the entropy after restriction to $\mathcal{H}_t(v, a)$ and renormalization. Exact ties were resolved deterministically without access to the withheld reference answer.

The inner model expressed v_t^* as a natural-language question without changing its graph-defined meaning. After answer a_t , the controller removed incompatible hypotheses and renormalized the remaining weights:

$$b_{t+1}(h) = \frac{\mathbf{1}[h \in \mathcal{H}_t(v_t^*, a_t)] b_t(h)}{\sum_{h' \in \mathcal{H}_t} \mathbf{1}[h' \in \mathcal{H}_t(v_t^*, a_t)] b_t(h')}. \quad (4)$$

If no hypothesis remained, the controller recorded an unresolved state rather than substituting a model-generated hypothesis.

The controller evaluated termination after each answer. It stopped when the remaining hypotheses supported a task output, no unasked question could distinguish them further or the task’s interaction limit was reached. Clarification questions were not repeated.

External knowledge source. The external knowledge source bounded the hypotheses the Orchestrator could consider, the clarification questions it could ask and the evidence it could pass to the downstream LLM. It did not generate the final response.

The reference implementation stored the task-specific graphs in Neo4j and implemented entity resolution, hypothesis retrieval, question enumeration and evidence retrieval as read-only Cypher queries. Neo4j

was an implementation choice rather than a requirement of the framework. The following sections describe how the general workflow was instantiated for diagnosis and food safety. Additional construction, provenance and implementation details are provided in Supplementary Methods.

Task design

We instantiated the framework in two tasks that require information absent from the initial query: diagnosis ranking from incomplete symptom profiles and food-safety classification without the patient’s health condition. Each benchmark instance paired an ambiguous query with a complete reference case. In the agent condition, the controller received only the ambiguous query; a simulation oracle accessed the withheld field solely to answer the questions selected by the controller. In a live interaction, those answers would be provided by the user.

The framework was compared with direct prompting and Rephrase-and-Respond (RaR) [13]. Direct prompting asked the downstream LLM to answer the supplied query without clarification. RaR reformulated the supplied query but did not acquire new information. The agent actively elicited missing context before invoking the same downstream LLM. No LLM was trained or fine-tuned for this study.

The unit of analysis was the case–query instance. The diagnosis benchmark contained 1,034 instances, and the food-safety benchmark contained 487. The primary diagnosis outcome was exact agreement at rank 1. The primary food-safety outcome was the Matthews correlation coefficient (MCC). All reported analyses were calculated from frozen row-level outputs.

Symptom–diagnosis Task

In the diagnosis task, visible symptoms were observations, candidate diagnoses were hypotheses and unasked symptoms connected to those diagnoses were clarification variables. `HAS_SYMPTOM` relations defined both candidate support and the hypothesis partition associated with each question. `IS_A` relations were used only to identify a shared diagnostic family when the available evidence did not resolve a single diagnosis.

Each instance contained a reference diagnosis, a complete symptom profile and a partition of the symptoms into visible and withheld sets. The controller received only the visible symptoms. When it asked about a symptom, the oracle returned “yes” if that symptom occurred in the complete profile and “no” otherwise. After clarification, the downstream LLM returned up to five ranked diagnoses. The task measured retrieval of a reference diagnosis from incomplete synthetic evidence; it did not evaluate autonomous diagnosis, triage or patient care.

Dataset and cohort construction. The benchmark was derived from Synthea, an open-source patient simulator based on clinical and epidemiological models [21]. The source cohort was generated on 24 August 2025 (UTC) from Synthea commit `c807432` (`master-branch-latest`), with the CSV and symptom exporters enabled. The run simulated Florida residents aged 0–90 years using all disease modules, a 10-year history, generation seed 404 and reference date 24 August 2025. The run produced 4,884 synthetic patients.

Benchmark construction selected disorder-labelled records with recorded symptoms and retained 40 reference pathologies. Patients without a retained case were excluded, leaving 4,518 patients and 10,374 case–diagnosis instances. Patients, rather than individual case rows, were assigned to the training, validation and test partitions (3,163, 904 and 451 patients, respectively), so that a patient could not occur in more than one partition. For each instance, a non-empty subset of the complete symptom profile was randomly withheld while at least one symptom remained visible. The withheld set was stored separately and was

available only to the simulation oracle. Complete provenance, partition counts and file hashes are provided in the Supplementary Methods.

The fixed evaluation cohort comprised 1,034 case–diagnosis instances from 451 synthetic patients and covered 32 reference diagnoses. The complete cases contained 8,503 symptom occurrences: 4,299 (50.6%) were initially visible and 4,204 (49.4%) were withheld. Patient identifiers did not overlap across the supplied training, validation and test partitions. No LLM was fitted on these partitions, and the held-out test partition was not used to construct the knowledge graph.

DxSxKG construction. The symptom–diagnosis knowledge graph (DxSxKG) represented diagnoses and symptoms as concepts from the 2025AA release of the Unified Medical Language System (UMLS) [23]. Diagnosis nodes were connected to symptom nodes by `HAS_SYMPTOM` relations and to clinical ancestors by `IS_A` relations. The evaluated graph contained 129,815 diagnosis nodes, 340,842 symptom nodes, 82,813 `HAS_SYMPTOM` relations and 1,400 `IS_A` relations. The relations used by the framework were obtained from UMLS and normalized to the schema described in Supplementary Methods. The held-out test partition did not contribute relations to DxSxKG.

Candidate construction. Candidate construction began by mapping each visible symptom mention to DxSxKG. Before mapping, the inner model marked mentions whose wording could support more than one clinical interpretation. The controller then searched normalized concept names, stored synonyms and the graph full-text index, in that order. If these searches returned no match, the inner model could propose canonical UMLS wording, which the controller accepted only when it matched an eligible graph node exactly. The resolver retained up to three graph matches for a short or ambiguity-flagged mention and the highest-ranked match for any other mention. Mentions that could not be resolved were excluded from candidate construction. Neither the reference diagnosis nor the withheld symptoms were available during mapping.

The resolved mentions were used for an initial diagnosis search. The inner model then made one expansion decision: whether a clinically plausible interpretation of the visible query was absent from the retrieved candidates. If so, it could propose up to three additional symptom terms. The controller discarded any proposed term that did not resolve to DxSxKG. In the evaluated configuration, graph-matched expansion terms were used together with the query-derived concepts to retrieve the final candidates and were not asked again as clarification questions. Expansion was not repeated after clarification began and did not use the withheld case information.

Let $\mathcal{P}$ denote the graph concepts obtained from the visible query and the single expansion step. When $|\mathcal{P}| = 1$, a diagnosis was eligible if it was connected to that concept; otherwise, it had to be connected to at least two concepts in $\mathcal{P}$. The graph query returned at most eight diagnoses, prioritizing greater concept overlap, greater proportional coverage of $\mathcal{P}$ and fewer graph-connected symptoms. Fixed diagnosis-name exclusions were then applied without replacing excluded candidates.

Each remaining candidate h was ranked by

$$s(h) = 2k_h + \frac{k_h}{m} - 0.01n_h, \quad (5)$$

where k_h is the number of concepts in $\mathcal{P}$ matched by h , $m = |\mathcal{P}|$, and n_h is the number of symptoms connected to h in DxSxKG. The first two terms reward direct and proportional overlap with the mapped query, whereas the final term gives a small preference to more specific diagnoses. The ranked candidates formed the initial hypothesis set $\mathcal{H}_0$.

Clarification and stopping. The controller initially assigned equal weight to every diagnosis in $\mathcal{H}_0$. At each turn, it considered unasked symptoms connected to the diagnoses that still had non-zero weight. Symptoms already represented in the query, previously answered or removed by the fixed question filter were not eligible. Among the remaining symptoms, the controller selected the one with the greatest positive expected information gain, using the rule defined above. The simulation oracle answered from the complete symptom profile. A “yes” answer removed diagnoses not connected to that symptom, whereas a “no” answer removed diagnoses that were connected to it; the controller then renormalized the weights and selected the next question.

Clarification ended when no initial diagnosis was retrieved, the leading diagnosis had posterior probability greater than 0.95, no eligible symptom had positive information gain or five questions had been asked. With the uniform initial weights and hard consistency updates used here, the posterior threshold was reached when one diagnosis remained, producing ANSWER. If the controller instead stopped because no informative question remained or the question limit was reached, it searched for an eligible immediate ISA ancestor shared by at least $\lceil |\mathcal{H}_0|/2 \rceil$ of the initially retrieved diagnoses. A qualifying family produced PARTIAL; otherwise the controller returned ABSTAIN. Failure to retrieve an initial diagnosis also produced ABSTAIN. A diagnosis generated after abstention was retained only for counterfactual inspection and was not scored as an answered framework case.

Diagnosis outcomes. The primary outcome was exact agreement between the reference diagnosis and the first returned diagnosis. Recall within the first five returned diagnoses was also calculated. Exact agreement required case-insensitive equality after trimming and normalizing whitespace. For secondary concept-based analyses, returned and reference names were resolved to UMLS concepts and assigned the ICD-10 mappings associated with those concepts. Broader diagnostic categories were derived from the corresponding UMLS hierarchy. The authors checked the final mapping table before calculating these outcomes.

Selective outcomes used answered cases as the denominator and were accompanied by coverage. Overall outcomes used all 1,034 instances and scored abstentions as incorrect. Clarification outcomes included the initial and terminal hypothesis-set sizes, entropy reduction, number of questions, stopping reason and terminal-state frequency.

Food-safety Task

The food-safety task tested whether the framework could recover patient context omitted from a dietary question before a downstream LLM classified the request. Each instance paired an ambiguous food query with a withheld patient condition and a binary reference decision, OKAY or NOT_OKAY. In the agent condition, the controller initially received only the ambiguous query. The withheld condition was available only to the simulation oracle, which used it to answer clarification questions. A deployed system would obtain these answers from the user.

Dataset construction. The food-safety benchmark comprised 487 instances: 301 labelled OKAY and 186 labelled NOT_OKAY. Each instance also retained the corresponding fully specified query and reference explanation. The Ambiguity Injection Module created the ambiguous query by removing the phrase that specified the patient’s health context while leaving the food request and reference decision unchanged. The removed context was stored separately for oracle simulation.

FoodSafetyKG construction. FoodSafetyKG integrated the USDA FoodData Central full CSV release dated 18 December 2025 [?], UMLS 2025AA condition concepts and hierarchies, and 14 clinical practice

guidelines. FoodData Central supplied food composition and nutrient records, UMLS supplied normalized condition concepts and aliases, and the guidelines supplied the associations between food properties and risk or safety for specific patient conditions.

The evaluated graph contained 2,085,340 food-item nodes, 477 nutrient nodes, 27,094,028 `HAS_NUTRIENT` relations, 2,542,284 derived `HAS_PROPERTY` relations and 21,030,444 materialized food-item risk relations. It also represented food phrases, ingredients, food properties, patient conditions, condition families and aliases. Food phrases and food items were connected to ingredients, nutrients and food properties, which were linked to conditions by guideline-supported `RISKY_FOR` and `SAFE_FOR` relations. The complete FoodSafetyKG was available throughout the reported evaluation.

Candidate construction. For each query, the inner model identified one or more food terms from the visible text, and a deterministic extractor supplied a fallback term when needed. The controller used each term to search FoodSafetyKG. It first retrieved conditions connected by direct `RISKY_FOR` relations and, when no direct relation was found, traversed the permitted food-ingredient-property-condition path. Conditions returned for multiple food terms were combined and deduplicated. A patient condition could therefore enter the hypothesis set only through graph-supported food-risk evidence.

The inner model then made one expansion decision: whether a relevant interpretation of the visible food query was absent from the initial graph search. If so, it could propose up to three additional food-search terms. A proposed term affected retrieval only when it matched FoodSafetyKG and returned at least one graph-supported risk condition. Expansion was performed once, before clarification, and did not use the withheld patient condition. The union of the retrieved conditions was limited to 50 and formed the fixed initial hypothesis set $\mathcal{H}_0$.

Clarification and stopping. Each condition in $\mathcal{H}_0$ received equal initial weight. Condition-family relations in FoodSafetyKG defined questions that partitioned several hypotheses, and individual conditions defined leaf questions. At each turn, the controller considered both question types and selected the partition with the greatest expected information gain. The inner model worded the selected graph variable as a question for the patient but could not change the condition or family being queried.

A “yes” answer to a family question retained the conditions belonging to that family, whereas a “no” answer removed them. An answer to an individual-condition question retained or removed that condition in the same way. The controller renormalized the remaining weights after each answer and then selected the next question. When only one unasked condition remained, the controller asked about it directly rather than inferring that it was present.

Clarification ended when a food-associated risk condition was confirmed, all graph-supported risk conditions were denied, the leading hypothesis exceeded posterior probability 0.95 or no remaining question had positive information gain. Failure to retrieve an initial risk hypothesis ended clarification with limited evidence. The food controller had no fixed numerical turn limit; the finite graph-defined question set bounded the interaction.

Evidence and prediction. After clarification, the controller retrieved the graph paths supporting the confirmed or remaining risk conditions and checked whether a confirmed condition also had an applicable `SAFE_FOR` relation. Confirmation of a risk condition without a safety exception produced a symbolic `NOT_OKAY` hint, whereas denial of all candidate risks or an applicable safety exception produced an `OKAY` hint. No symbolic class was assigned when graph-supported clarification remained incomplete.

The inner model assembled the original query, disclosed answers, remaining conditions, stopping reason and graph evidence into a structured clarified record. It could organize this information but could not change

the selected questions, hypothesis updates, stopping decision or symbolic hint. The downstream LLM received this record and returned one binary prediction. Clarification completeness was recorded separately and was not treated as a third prediction class.

Food-safety outcomes. MCC was the primary discrimination measure because it incorporates all four cells of the confusion matrix under class imbalance. Accuracy, precision, recall and F_1 were also calculated with NOT_OKAY as the positive class, together with the false-negative and false-positive rates. The raw tokens OK and NOT_OK were normalized to OKAY and NOT_OKAY, respectively. A missing or invalid output was not assigned a class and was excluded from metrics requiring a valid binary prediction; the corresponding denominator was reported.

Clarification outcomes included the initial and terminal hypothesis-set sizes, entropy reduction, number and type of questions, stopping reason and resolution completeness. All valid binary outputs entered the primary classification analysis irrespective of whether clarification was complete.

Models, comparators and statistical analysis

The downstream panel comprised GPT-5.5 (gpt-5.5; OpenAI), Claude Opus 4.8 (claude-opus-4-8; Anthropic), Gemini 3.1 Pro Preview (google/gemini-3.1-pro-preview; a Google model served through OpenRouter), Llama 3.3 70B Instruct (meta-llama/llama-3.3-70b-instruct; OpenRouter) and Mistral Large (mistral-large-2512; Mistral AI). The requested identifiers, provider-returned identifiers when available and access timestamps were stored in the run manifests. GPT-5.5 was fixed as the inner clarification model; only the downstream response model varied across the model panel.

Direct prompting combined the supplied query with the task-specific output instruction. RaR used two calls [13]: the first reformulated the supplied query without adding information, and the second generated a response from the original and reformulated queries. RaR could not ask clarification questions or access the simulation oracle. In the agent condition, clarification preceded response generation and the downstream LLM received the clarified query, structured graph evidence and the task-specific output instruction.

For the diagnosis task, all three configurations began with the same incomplete symptom presentation. In the frozen food-safety evaluation, the agent began with the ambiguous food query and recovered patient context through simulated answers, whereas the direct and RaR prompts contained the fully specified query including patient context. The food-safety comparator results therefore represent context-available prompting controls, not matched-input ablations of the clarification procedure.

Prompt templates and output schemas were fixed across models within each task and configuration. Verbatim prompts, schemas, decoding settings, retry rules, parsers, prompt hashes and task-specific access timestamps are retained in the released configuration manifests; their reporting structure is summarized in Supplementary Methods.

Metrics were calculated at the case–query level from frozen row-level predictions. Configurations were aligned by benchmark instance. The primary comparisons were descriptive; no null-hypothesis significance tests or multiplicity adjustments were applied. Each model–configuration result was reported with its analysis denominator, missing outputs and abstentions. Analyses restricted to answered cases were designated as selective and were reported with coverage. Direct comparisons used either the complete cohort or an explicitly defined common-case subset.

Between-model dispersion was summarized across the five downstream models by the mean, standard deviation and range of each model-level metric. The five models were purposively selected; these summaries were therefore descriptive and were not interpreted as estimates for a population of language models.

Repeated generation was examined as an exploratory analysis of output stability. Two evaluation cases were selected from each task, and each model–configuration pair was sampled independently 500 times per case. In the agent condition, the recorded clarification trajectory and clarified query were held fixed so that only downstream response generation was resampled.

For diagnosis, predictive entropy was calculated over normalized exact diagnosis strings and, separately, over mapped diagnostic categories. For observed outputs with empirical frequencies $p(y)$, entropy was

$$H(Y) = - \sum_y p(y) \log_2 p(y). \quad (6)$$

For food safety, the same calculation was applied to the two normalized binary labels. Lower entropy indicated greater output consistency but did not indicate correctness. Because only two cases per task were evaluated, these analyses were interpreted as case studies rather than population-level stability estimates.

Reproducibility and ethical considerations

The framework was implemented in Python within openCHA [30]. Task-specific graphs were accessed through the official Neo4j Python driver using read-only Cypher queries. No LLM was trained or fine-tuned, and no GPU was required to recalculate the reported metrics from the frozen outputs.

Commercial model calls cannot be reproduced byte-for-byte because provider infrastructure and served aliases may change. We therefore froze the requested model identifiers, prompts, structured evidence, outputs and SHA-256 hashes used for the reported analyses. The offline workflow verifies these artifacts and recalculates the tables without network, API or database access. A new live model call is treated as a replication attempt rather than exact regeneration.

The study involved no participant recruitment, clinical intervention or deployment in patient care. The diagnosis task used synthetic patient records, and the oracle responses in both tasks were simulated from fields already contained in the benchmarks. The evaluation therefore does not establish clinical safety, effectiveness, fairness or usability with real patients. The framework should not be interpreted as an autonomous diagnostic or dietary decision system.

Data availability

The symptom–diagnosis benchmark was derived from synthetic Synthea records and contains no real patient records. The food-safety benchmark was constructed for this study from authored dietary-query templates and the rule sources described in the manuscript. Frozen benchmark inputs and row-level predictions will be made available with the peer-review release, subject to confirmation of the final redistribution and citation metadata. UMLS Metathesaurus source files are available separately from the US National Library of Medicine under the UMLS license and are not redistributed by the authors.

Code availability

Source code for constructing the knowledge graphs, running the ambiguity experiments and reproducing the reported metrics will be available in the versioned project release. The release includes a pinned Python environment, frozen row-level predictions, SHA-256 manifests and an offline reproduction command. Licensed UMLS files and commercial API credentials are not included.

References

- [1] Draelos, R. L. *et al.* Large language models provide unsafe answers to patient-posed medical questions. *npj Digital Medicine* **9**, 241 (2026).
- [2] Piantadosi, S. T., Tily, H. & Gibson, E. The communicative function of ambiguity in language. *Cognition* **122**, 280–291 (2012).
- [3] Min, S., Michael, J., Hajishirzi, H. & Zettlemoyer, L. AmbigQA: Answering ambiguous open-domain questions. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 5783–5797 (2020).
- [4] Min, S., Lee, K., Chang, M.-W., Toutanova, K. & Hajishirzi, H. Joint passage ranking for diverse multi-answer retrieval. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 6997–7008 (2021).
- [5] Stelmakh, I., Luan, Y., Dhingra, B. & Chang, M.-W. ASQA: Factoid questions meet long-form answers. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 8273–8288 (2022).
- [6] Lewis, P. *et al.* Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*, 9459–9474 (2020).
- [7] Gao, Y. *et al.* Answering ambiguous questions through generative evidence fusion and round-trip prediction. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (ACL-IJCNLP)*, 3263–3276 (2021).
- [8] Zhang, T. *et al.* CLAMBER: A benchmark of identifying and clarifying ambiguous information needs in large language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 10746–10766 (2024).
- [9] Kim, H. J. *et al.* Aligning language models to explicitly handle ambiguity. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 1989–2007 (2024).
- [10] Liu, L. *et al.* A survey on large language models from general purpose to medical applications: Datasets, methodologies, and evaluations. *arXiv preprint arXiv:2406.10303* (2024).
- [11] Amugongo, L. M., Mascheroni, P., Brooks, S., Doering, S. & Seidel, J. Retrieval augmented generation for large language models in healthcare: A systematic review. *PLOS Digital Health* **4**, e0000877 (2025).
- [12] Rezaei, M. R., Fard, R. S., Parker, J. L., Krishnan, R. G. & Lankarany, M. Agentic medical knowledge graphs enhance medical question answering: Bridging the gap between LLMs and evolving medical knowledge. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, 12682–12701 (2025).
- [13] Deng, Y., Zhang, W., Chen, Z. & Gu, Q. Rephrase and respond: Let large language models ask better questions for themselves (2024). 2311.04205.

- [14] Savage, T. *et al.* Large language model uncertainty proxies: discrimination and calibration for medical diagnosis and treatment. *Journal of the American Medical Informatics Association* **32**, 139–149 (2025).
- [15] Liu, Y. *et al.* Mind the ambiguity: Aleatoric uncertainty quantification in LLMs for safe medical question answering. In *Proceedings of the ACM Web Conference (WWW)* (2026). ArXiv:2601.17284.
- [16] Kuhn, L., Gal, Y. & Farquhar, S. CLAM: Selective clarification for ambiguous questions with large language models (2023). 2212.09748.
- [17] Lee, D. *et al.* Asking clarification questions to handle ambiguity in open-domain QA. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, 11526–11544 (2023).
- [18] Chen, Y. *et al.* STYLE: Improving domain transferability of asking clarification questions in large language model powered conversational agents. In *Findings of the Association for Computational Linguistics: ACL 2024*, 10633–10649 (2024).
- [19] Li, S. S. *et al.* MediQ: Question-asking LLMs and a benchmark for reliable interactive clinical reasoning. In *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)* (2024).
- [20] Wen, L. *et al.* CLEAR-KGQA: Clarification-enhanced ambiguity resolution for knowledge graph question answering (2025). 2504.09665.
- [21] Walonoski, J. *et al.* Synthea: An approach, method, and software mechanism for generating synthetic patients and the synthetic electronic health care record. *Journal of the American Medical Informatics Association* **25**, 230–238 (2018).
- [22] Chen, J., Chun, D., Patel, M., Chiang, E. & James, J. The validity of synthetic clinical data: a validation study of a leading synthetic data generator (Synthea) using clinical quality measures. *BMC Medical Informatics and Decision Making* **19**, 44 (2019).
- [23] Bodenreider, O. The unified medical language system (UMLS): integrating biomedical terminology. *Nucleic Acids Research* **32**, D267–D270 (2004).
- [24] Liu, A. *et al.* We’re afraid language models aren’t modeling ambiguity. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 790–807 (2023).
- [25] Tamkin, A., Handa, K., Shrestha, A. & Goodman, N. Task ambiguity in humans and language models (2023). ICLR 2023, 2212.10711.
- [26] Cole, J. R. *et al.* Selectively answering ambiguous questions. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 530–543 (2023).
- [27] Kuhn, L., Gal, Y. & Farquhar, S. Semantic uncertainty: Linguistic invariances for uncertainty estimation in natural language generation. In *International Conference on Learning Representations (ICLR)* (2023).
- [28] Xiong, M. *et al.* Can LLMs express their uncertainty? an empirical evaluation of confidence elicitation in LLMs. In *International Conference on Learning Representations (ICLR)* (2024).
- [29] Lin, Z., Trivedi, S. & Sun, J. Generating with confidence: Uncertainty quantification for black-box large language models. *Transactions on Machine Learning Research* (2024).

- [30] Abbasian, M., Azimi, I., Rahmani, A. M. & Jain, R. Conversational health agents: a personalized large language model-powered agent framework. *JAMIA Open* **8**, ooaf067 (2025).
- [31] Neumann, M., King, D., Beltagy, I. & Ammar, W. ScispaCy: Fast and robust models for biomedical natural language processing. In *Proceedings of the 18th BioNLP Workshop and Shared Task*, 319–327 (2019).
- [32] Eyre, H. *et al.* Launching into clinical space with medspaCy: a new clinical text processing toolkit in Python. *AMIA Annual Symposium Proceedings* **2021**, 438–447 (2021).
- [33] Miller, G. A. WordNet: A lexical database for English. *Communications of the ACM* **38**, 39–41 (1995).

Author contributions

Competing interests

A Supplementary Methods

These Supplementary Methods provide the implementation, provenance and prompt details required to interpret and reproduce the experiments reported in the main manuscript. The analysis cohorts, task definitions and primary outcomes are identical to those in the main Methods. Licensed UMLS files and commercial API credentials are not redistributed.

A.1 Diagnosis benchmark construction

The diagnosis benchmark was generated with Synthea [21], using commit `c807432`, Florida as the geographic setting, ages 0–90 years, all disease modules, a 10-year history and a reference date of 24 August 2025. CSV and symptom exporters were enabled. The patient-generation seed was 404, the clinician seed was 1755999126513 and the final run was recorded at `2025-08-24T01:32:06Z`. It used Java 17.0.14 and generated 4,884 synthetic patients.

Benchmark construction retained disorder-labelled pathology episodes with at least one symptom. Parenthetical Synthea qualifiers were removed from labels, and duplicate records were collapsed to one case–diagnosis instance for each patient and retained pathology. Forty pathologies were retained across the complete benchmark. Of the generated patients, 4,518 contributed at least one instance; 366 contributed none and were excluded.

Patients were partitioned before evaluation. The training, validation and test partitions contained 3,163, 904 and 451 patients, respectively, with no patient overlap. For each instance, a non-empty subset of symptoms was withheld while at least one symptom remained visible. The test partition contained 1,034 instances and 32 reference diagnoses. Its 8,503 symptom occurrences comprised 4,299 visible occurrences (50.6%) and 4,204 withheld occurrences (49.4%).

Supplementary Table 1: Flow of synthetic patients and case–diagnosis instances into the diagnosis benchmark.

Stage or partition	Patients	Instances
Generated by Synthea	4,884	–
Excluded: no retained benchmark instance	366	–
Included in the frozen benchmark	4,518	10,374
Training partition	3,163	7,258
Validation partition	904	2,082
Test partition	451	1,034

The partition-file SHA-256 hashes were:

Training: 51960aa4b1fbf6a94bd0bd72c6cbab65fd4c04b313768d0d4a259ca84d990573

Validation: 7eeb5a0226f39834a3ca74a321e73fbc0e5a01ab747c981d17920be9dc5c0fa3

Test: c2eaa6dd06ca784e5748344ed6f4be43a2925eedc166d2ef48db91028eeb28.

These files preserve the evaluated assignments and symptom partitions even if the original randomization is not rerun.

A.2 DxSxKG construction and concept alignment

DxSxKG was constructed from UMLS 2025AA [23], using MRCONSO for names and synonyms, MRSTY for semantic types and MRREL for relations. The held-out test partition did not contribute graph relations. Each node was keyed by its concept unique identifier (CUI) and retained its preferred name, synonyms, semantic type and source vocabulary.

Diagnosis nodes were concepts assigned T047 (Disease or Syndrome) or T048 (Mental or Behavioral Dysfunction). Symptom nodes were concepts assigned T184 (Sign or Symptom) or T033 (Finding). Concepts satisfying both definitions were represented as diagnoses. Direct diagnosis–manifestation relations were normalized to HAS_SYMPTOM; reverse-direction relations were reoriented, suppressed records were excluded and duplicate pairs were collapsed. Original relation labels were retained as provenance.

Clinical-family relations were stored as IS_A edges. They supported only the family-level terminal state and did not affect candidate retrieval, question selection or belief updating.

Supplementary Table 2: DxSxKG artifact used in the reported evaluation.

Component	Count
Diagnosis nodes	129,815
Symptom nodes	340,842
HAS_SYMPTOM relations	82,813
IS_A relations	1,400

Visible symptoms were normalized by lowercasing, collapsing whitespace and removing parenthetical qualifiers. Resolution then attempted exact-name, stored-synonym and full-text matching. If these steps failed, the inner model could propose a UMLS label, which the controller accepted only when it exactly matched an eligible graph node. Unresolved strings did not contribute to candidate retrieval. The reference diagnosis and withheld symptoms were unavailable during resolution.

The UMLS archive was `umls-2025AA-metathesaurus-full.zip`; its source files were timestamped 28 April 2025. Their SHA-256 hashes were:

```
MRCONSO.RRF: bfb75116adc9f0308025ea90c05b5c356896198d307687b820979e6bf782f058
MRREL.RRF: 44d896e9ff8046b38a0ebd15e40615314e12ef37d0d99901bb3976014b1cbfd7
MRSTY.RRF: 2712ae33d6450492a0415ea9d3291dc6cee6e0b19b89df7081044515b69f227a.
```

A.3 Diagnosis controller implementation

The frozen controller was identified as `diagnostic-agent-release-v1-compatible-2026-07`. A short or ambiguity-flagged mention could retain up to three graph resolutions; another mention retained the highest-ranked resolution. The inner model then made one expansion decision and could propose up to three additional symptom terms. A proposed term entered retrieval only after resolution to an eligible graph concept. Expansion was not repeated and could not access withheld symptoms.

Let $\mathcal{P}$ denote the retained concepts. A diagnosis required one match when $|\mathcal{P}| = 1$ and at least two otherwise. The graph returned at most eight diagnoses, ordered by decreasing match count, decreasing match fraction and increasing number of connected symptoms. Fixed diagnosis-name exclusions were then applied without replacement.

The controller assigned equal initial weight to each diagnosis. At each turn it selected the unasked symptom with greatest positive expected information gain among diagnoses with non-zero posterior weight. The oracle answered from the complete synthetic symptom profile. A positive answer retained diagnoses linked to the symptom; a negative answer removed them. Remaining weights were renormalized.

Clarification stopped when one diagnosis exceeded posterior probability 0.95, no question had positive information gain or five questions had been asked. A unique remaining diagnosis produced `ANSWER`. Otherwise, an eligible immediate `ISA` family covering at least $\lceil |\mathcal{H}_0|/2 \rceil$ initially retained diagnoses produced `PARTIAL`; absence of such a family produced `ABSTAIN`.

A.4 Food-safety benchmark construction

The food-safety benchmark contained 487 instances: 301 labelled `OKAY` and 186 labelled `NOT_OKAY`. Each record preserved the fully specified query, ambiguous query, removed patient context, reference label and reference explanation. In the Agent condition, the removed context was accessible only to the oracle and was not included in food extraction, graph expansion, question selection or downstream input before disclosure through an answer.

A.5 FoodSafetyKG construction and provenance

FoodSafetyKG integrated the USDA FoodData Central full CSV release dated 18 December 2025 [?], UMLS 2025AA condition concepts and hierarchies, and clinical-guideline rules. FoodData Central supplied food composition and nutrient records. UMLS supplied normalized condition concepts, aliases and parent-child relations. The guideline registry supplied food properties, thresholds and condition-specific risk or safety associations. Each authored or derived relation retained its source and evidence type.

The graph represented food phrases, USDA food items, ingredients, nutrients, food properties, conditions, condition families and aliases. Its principal evidence paths connected food phrases through ingredients and properties to conditions, and food items through nutrients or properties to conditions. Materialized RISKY_FOR and SAFE_FOR relations shortened these paths without discarding provenance. The complete FoodSafetyKG was available throughout the reported evaluation.

Supplementary Table 3: FoodSafetyKG components used in the reported evaluation.

Component	Count
USDA food-item nodes	2,085,340
Nutrient nodes	477
Food-phrases nodes	298
Ingredient nodes	130
Food-property nodes	60
Condition nodes	67
Condition-family nodes	16
Condition-alias nodes	138
HAS_NUTRIENT relations	27,094,028
Derived food-item HAS_PROPERTY relations	2,542,284
Materialized food-item risk relations	21,030,444
Curated food-phrase-ingredient relations	398
Curated ingredient-property relations	341
Property-condition RISKY_FOR relations	126
Property-condition SAFE_FOR relations	67

The FoodData Central files were extracted from FoodData_Central_csv_2025-12-18.zip. Their SHA-256 hashes were:

```

food.csv: a1ec0c8ebee04f91de8c9549486f555f3a8367e538a7a6da4dbb0f068f76a983
food.nutrient.csv:
e4085bacf4e986304f01545e90a2b6ab5a30f204395a471db4d9854cd8cd74ed
nutrient.csv: 226ba937d1a73e8c87bd7fdcd577524699bef4025c9232ee474b7d3083498ff.

```

The rule registry grouped its principal clinical sources into ADA, AHA, ACG, NKF KDOQI, ACR, FARE, ATA, ACOG, NOF, AASLD, AHS, FDA, WHO and ESC guideline families. The machine-readable registry records issuing body, year, source URL, evidence level and affected property-condition relation.

A.6 Food-safety controller implementation

The frozen controller was identified as `food-agentic-pomdp-v4-shared-stopping`. The inner model extracted one or more food terms from the visible query; a deterministic extractor provided a fallback term. The controller queried direct `RISKY_FOR` relations first and used the permitted food-ingredient-property-condition path when direct retrieval returned no condition. Results from all accepted terms were combined, deduplicated and limited to 50 conditions.

The inner model then made one expansion decision and could propose up to three additional food, ingredient, synonym or category terms. A proposed term affected retrieval only after graph resolution and return of at least one supported risk condition. Expansion was performed once and could not access the hidden patient context. It added no condition in the 487 frozen cases.

Each condition received equal initial weight. Condition families reachable through one to three `IS_A` edges defined family questions; individual conditions defined leaf questions. The controller selected the question with greatest expected information gain. Exact ties favoured a family question and then the normalized question name. When one unasked condition remained, the controller asked it directly.

Positive family answers retained member conditions; negative answers removed them. Leaf answers retained or removed one condition. Clarification ended after confirmation of a risk condition, denial of all graph-supported risks, posterior probability greater than 0.95 or exhaustion of positive-information-gain questions. There was no numerical turn limit; the finite question pool bounded the interaction.

After clarification, the controller retrieved supporting graph paths and checked applicable `SAFE_FOR` relations. A confirmed risk without a safety exception produced a symbolic `NOT_OKAY` hint. Denial of all retrieved risks or an applicable safety exception produced an `OKAY` hint. Incomplete clarification produced no symbolic class. The inner model could organize the evidence record but could not change its symbolic fields.

The frozen Stage-A artifact contained all 487 cases and had SHA-256 hash

```
ef34b7872048178e313d52d730228322e4328b9117b7fe1a128b95b25899d1cc.
```

Clarification completed in 485 cases; two had no graph-supported initial hypothesis. The mean was 5.96 questions per case and the maximum was 11. Every case was passed to the downstream model for one binary prediction.

A.7 Model routing and inference configuration

The inner clarification model was GPT-5.5 through the OpenAI API. The same five downstream models were used in both tasks. Model developer and API provider were recorded separately because Gemini and Llama were served through OpenRouter.

Supplementary Table 4: Model routing used for the frozen evaluation.

Display label	Developer	API provider	Requested identifier
GPT-5.5	OpenAI	OpenAI	gpt-5.5
Claude Opus 4.8	Anthropic	Anthropic	claude-opus-4-8
Gemini 3.1 Pro	Google	OpenRouter	google/gemini-3.1-pro-preview
Llama 3.3 70B Instruct	Meta	OpenRouter	meta-llama/llama-3.3-70b-instruct
Mistral Large	Mistral AI	Mistral	mistral-large-2512

Prompts and output schemas were fixed within each task and configuration. Provider-returned identifiers, request timestamps, output-token limits, retry outcomes, raw outputs and parsed outputs were stored when available. The frozen food manifests record runs for Claude, Gemini, Llama and Mistral on 20 July 2026 (UTC). A new commercial-model call is a replication rather than an exact regeneration because a served alias or stochastic output can change.

For diagnosis, all configurations received the same incomplete symptom presentation. For food safety, the frozen Direct and RaR prompts contained the fully specified query, including patient context, whereas the Agent began with the ambiguous query. The food baselines are therefore context-available prompting controls.

B Prompt templates and output schemas

The templates below document the instruction roles, decision constraints and output schemas that affected the evaluated outputs, allowing reviewers to separate model behaviour from deterministic graph operations. Angle-bracketed fields were populated at runtime. Exact machine-readable strings, model identifiers, decoding settings and prompt hashes should accompany the released evaluation records. Deterministic graph queries, information-gain calculations and belief updates did not use prompts.

B.1 Direct and Rephrase-and-Respond prompts

Supplementary Prompt 1 — Direct diagnosis output

<INCOMPLETE SYMPTOM CONVERSATION>

You are a medical assistant helping to determine a likely diagnosis. Provide your top 5 possible diagnoses ranked from most likely to least likely.

Return ONLY valid JSON with this exact schema:

```
{'diagnoses': [{ 'name': 'diagnosis name', 'icd10': 'ICD-10-CM code', ...}]}
```

Provide exactly 5 entries. For each, give the diagnosis name and its standard ICD-10-CM code.

Supplementary Prompt 2 — Direct food-safety output

<FULLY SPECIFIED FOOD QUERY>

The question asks whether it is OK for the patient to eat/drink/do the thing described, given their context. Answer with exactly one token: OK or NOT_OK. Do not explain.

RaR used two calls. The first received:

Supplementary Prompt 3 — RaR reformulation call

``<SUPPLIED QUERY>``

Rephrase and expand the question, and respond.

The second call received:

Supplementary Prompt 4 — RaR answer call

(original) <SUPPLIED QUERY>

(rephrased) <REFORMULATED QUERY>

Use your answer for the rephrased question to answer the original question.

<TASK-SPECIFIC OUTPUT INSTRUCTION FROM PROMPT 1 OR 2>

B.2 Diagnosis inner-model prompts

Supplementary Prompt 5 — UMLS fallback resolver

You are a medical terminology expert.
Map this patient symptom to the closest UMLS concept name in a medical knowledge graph.
Patient symptom: ``<SYMPTOM>``
Sample concept names from the knowledge graph:
<UP TO 40 GRAPH LABELS>
Rules:
1. Return ONLY the concept name -- no explanation.
2. If the symptom matches a sample name exactly, return that.
3. Otherwise return the closest medical equivalent.
4. Keep it under 6 words.

Supplementary Prompt 6 — Diagnosis question wording

Rephrase the medical symptom ``<GRAPH SYMPTOM NAME>`` as a simple yes/no question for a patient. Return only the question. Under 15 words.

B.3 Diagnosis downstream-model prompts

Supplementary Prompt 7 — Diagnosis ANSWER state

You are a board-certified physician acting as a diagnostic verifier.
The symbolic POMDP agent selected one diagnosis with high confidence.
Verify it is clinically plausible given the evidence summary.
If plausible: confirm it.
If NOT plausible: choose the best alternative from the candidate set ONLY.
If the top candidate is in the candidate set and matches the evidence, always confirm it.
Return ONLY valid JSON:
{ ``final\_diagnosis``: ``exact candidate name``, ``decision``:
``confirmed``, ``prediction\_top5``: [``d1``, ``d2``, ``d3``, ``d4``, ``d5``],
``reason``: ``one evidence-based sentence`` }

Supplementary Prompt 8 — Diagnosis PARTIAL state

You are a board-certified physician acting as a diagnostic ranker. The symbolic agent identified a clinical family but cannot distinguish between candidates. Rank the candidates using the confirmed and denied evidence.

Rules:

- Choose ONLY from the candidate set.
 - Use confirmed and denied symptoms as primary evidence.
 - Use prevalence only as a tiebreaker.
 - Prefer viral to bacterial sinusitis unless purulent discharge, duration over 7 days, severe focal facial pain or double worsening is present.
 - When diarrhea, productive cough or wheezing is confirmed, prefer supported lower-respiratory diagnoses over upper-respiratory diagnoses.
 - When symptoms are limited to throat, tonsils or mild nasal symptoms, prefer supported upper-respiratory diagnoses.
- Return ONLY valid JSON with final_diagnosis, a five-item prediction_top5 and one evidence-based reason.

The post-abstention prompt supported counterfactual inspection only. Its output was not scored as an answered framework case.

B.4 Food-safety inner-model prompts

Supplementary Prompt 9 — Food-query ambiguity analysis

You are a clinical food-query ambiguity analyst. Analyse only the visible patient query. Do not invent a diagnosis or restriction.

Patient query: ``<VISIBLE QUERY>``

Check for food-identity, ingredient, preparation, portion, patient-context and timing ambiguity.

Provide simple lowercase food, ingredient and useful generic-category terms for KG retrieval. Include compound and component terms when both are visible, but do not add foods not mentioned.

Return ONLY this JSON:

```
{``ambiguity-types``: [``type``], ``food-terms``: [``term``],  
  ``missing_context``: [``item``], ``multi-interpretation-needed``:  
  [``term``], ``summary``: ``one sentence``}
```

Supplementary Prompt 10 — One-time food-term expansion

You are evaluating a food-safety KG hypothesis set.

Visible query: <VISIBLE QUERY>

Ambiguity report: <AMBIGUITY JSON>

Food terms already searched: <TERMS>

Current risky-condition hypotheses: <CONDITIONS>

Expand if the set is empty, has fewer than three hypotheses, or is clinically incoherent with the visible food. Suggest at most three food, ingredient, synonym or generic-category search terms. Do not suggest diagnoses and do not use undisclosed patient context.

Return ONLY this JSON:

```
{``needs_expansion``: false, ``reason``: ``one sentence``,  
  ``additional_terms``: []}
```

Supplementary Prompt 11 — Food question wording

For a leaf condition: Rephrase the medical condition name into a simple yes/no patient question. Return ONLY the question, without explanation.
For a family: Rephrase this condition-family screen into a simple yes/no patient question asking whether the patient has any condition in that family. Return ONLY the question, without explanation.

Supplementary Prompt 12 — Structured food evidence record

You are a clinical evidence synthesiser. Build the structured food-safety evidence bundle u-prime for a final model. Use only the visible query, disclosed clarification answers and supplied KG evidence. Do not infer an undisclosed patient condition. Do not change clarification completeness or any supplied symbolic binary hint.

Inputs: visible query; ambiguity report; food terms; disclosed Q&A; remaining conditions; risky and safe KG evidence; food properties; resolution status; condition family; symbolic binary hint.

Return ONLY JSON with these keys: presentation, ambiguity_resolved, foodinterpretation, confirmedevidence, ruledout, never_asked, remaining_uncertainty, resolution_complete, condition_family, candidate_conditions, food_properties, symbolic_binary_hint and safety_question.

Before downstream generation, the controller restored the symbolic values for resolution completeness, condition family, candidate conditions, food properties and the binary hint. The evidence-synthesis model could not alter these fields.

B.5 Food-safety downstream-model prompts

Supplementary Prompt 13 — Completed food clarification

You are a clinical dietitian verifying a binary food-safety decision. The symbolic ambiguity agent asked clarification questions and completed its path. Use only disclosed answers, the food query and supplied KG evidence. Do not assume undisclosed conditions.

Distinguish direct contraindications from conditional or moderation risks. Treat raw fish or undercooked meat in pregnancy, alcohol in pregnancy or liver disease, gluten in celiac disease, lactose in lactose intolerance, strongly high-sodium processed foods in kidney disease or hypertension, directly applicable stimulant/cafeine or GERD triggers, and high-sugar drinks or desserts in diabetes as direct risks when supported by the supplied evidence.

Do not treat a general property as an absolute contraindication when the food wording, preparation or portion makes the relation conditional.

Foods such as rice, oatmeal, fruit, whole-grain bread and salmon may be acceptable in moderation unless a direct relation applies to the exact item. Respect modifiers such as gluten-free and low-sodium.

Return ONLY valid JSON:

```
{``decision``: ``okay or not.okay``, ``confidence``: ``high or medium or low``, ``reasoning``: ``one evidence-based sentence``}
```

Supplementary Prompt 14 — Limited food clarification

You are a clinical dietitian making the required binary food-safety judgment after a bounded clarification process. Use only the visible query, disclosed clarification answers, remaining KG hypotheses, ingredients, preparation and food properties. Do not assume an undisclosed condition. Use low confidence when essential information remains missing.

Return ONLY valid JSON:

```
{``decision``: ``okay or not.okay``, ``confidence``: ``high or medium or low``, ``reasoning``: ``one sentence``}
```

C Outcome processing and reproducibility

C.1 Diagnosis outcomes

Exact Top-1 and exact Recall@5 required literal agreement after case folding, trimming and whitespace normalization. Secondary analyses resolved returned and reference names to UMLS concepts and used their ICD-10 mappings. Broader categories were derived from the UMLS hierarchy. The authors checked the final mapping table before calculating the outcomes.

Selective outcomes used the 999 answered instances as the denominator and were accompanied by coverage. Overall outcomes used all 1,034 test instances and scored abstentions as incorrect. A diagnosis generated for counterfactual inspection after ABSTAIN was not counted as an answered output.

C.2 Food-safety outcomes

Raw OK and NOT_OK tokens were normalized to OKAY and NOT_OKAY. NOT_OKAY was the positive class. All valid binary predictions entered the primary analysis, including the two cases in which clarification ended with limited evidence. Missing or invalid outputs were retained as missing and were not assigned a class.

C.3 Frozen artifacts and offline verification

The release contains the 30 primary prediction files formed by two tasks, five downstream models and three configurations. A SHA-256 manifest records the byte size and hash of each artifact. The offline workflow verifies the manifest and recalculates the reported metrics without API, database, UMLS or USDA access. Live runners write to a separate directory and do not overwrite the frozen release.

Exact reproduction means recalculation from the frozen inputs and outputs. A new commercial-model call is a replication because the provider can update a served alias and stochastic generation can return another valid response.

D Repeated-generation analyses

Two cases were selected from each task. Each model–configuration pair was sampled 500 times per case. In the Agent condition, the recorded clarification trajectory, terminal state and clarified query were held fixed; only the downstream response was regenerated. Direct and RaR regenerated their complete responses on each repetition.

For diagnosis, predictive entropy was calculated over normalized exact diagnosis strings and, separately, mapped diagnostic categories. For food safety, entropy was calculated over the two binary labels. For empirical output frequencies $p(y)$,

$$H(Y) = - \sum_y p(y) \log_2 p(y). \quad (7)$$

Lower entropy indicates greater consistency, not greater accuracy. These analyses characterize four selected cases and do not estimate stability over either complete benchmark. Because the Agent trajectory was fixed, they do not measure end-to-end variability in question selection or simulated answers.

E Scope of the controlled evaluation

The evaluation used synthetic or authored cases and simulated answers. It did not test patient comprehension, contradictory or uncertain answers, clinician–AI interaction, treatment recommendations, workflow integration or prospective outcomes. The diagnosis task measured retrieval of a recorded synthetic diagnosis from incomplete symptoms; the food task measured binary classification within the represented food and condition relations. Neither task evaluated an autonomous clinical decision system.

The framework inherits the coverage and accuracy limits of its graphs. A missing entity or relation can prevent retrieval of a relevant hypothesis or question. A supported relation can also be too general for an individual because preparation, portion, disease severity, medication use and comorbidity may alter its applicability. These limits motivate evaluation with independently reviewed cases, noisy answers and naturally occurring queries.

F Ambiguity Injection Module

Clarification agents require many examples in which the user’s surface utterance is intentionally incomplete, yet the information needed to resolve that incompleteness is known. We address this gap with the *Ambiguity Injection Module* (AIM), an offline generator that transforms *fully specified* seed queries into *underspecified* variants while retaining the original intent as latent structure. Conceptually, AIM instantiates an *inverse slot-filling* process: standard natural-language understanding maps text to filled slots; AIM begins from text that already encodes those slots and applies controlled transformations that remove, blur, or rephrase slot content on the surface while preserving a recoverable target for clarification. The two evaluation sets in this study were produced with the rule-based masking track described below, applied to structured slots (Synthea symptom lists [21] and the patient-context phrase of curated dietary queries); the remaining tracks are part of the module and are described for completeness.

F.1 Entity grounding

AIM first identifies domain-relevant spans that can be manipulated as slots. For mental-health-oriented text, we combine clinical NLP tagging (scispaCy [31], medspaCy [32]) with lightweight pattern constraints so that symptoms, temporal cues, severity markers, and related phrases can be addressed consistently. For nutrition-oriented text, we parse ingredients, amounts, and units from free-form recipe or meal descriptions. Where the seed is already structured, as in the benchmarks used here, slot boundaries are taken directly from the source fields. This layer does not perform downstream diagnosis or safety reasoning; it only supplies reliable anchors for ambiguity operations.

F.2 Complementary ambiguity mechanisms

AIM composes three families of transformations that target different linguistic sources of underspecification.

Rule-based masking simulates omission and abstraction: entities or phrases are dropped, quantities are replaced with vague scoping, and domain-specific combinations of cues can be removed jointly. In mental-health scenarios, we distinguish *safety-preserving* transformations from naive deletion: high-risk content is not simply erased when doing so would remove clinically salient signals; instead, selected terms are mapped to vaguer surface forms so that downstream models still encounter risk-relevant language while remaining underspecified for safe decision-making.

Ontological generalization replaces specific expressions with broader categories by traversing lexical hypernym structure (WordNet [33]) over short hop distances, supplemented by small domain lexicons where WordNet coverage is thin. This yields imprecision that remains plausible in user language (e.g., a specific food → a coarse food class) without collapsing to trivial supersenses.

Persona-conditioned rewriting uses a large language model instructed to imitate hesitant or novice users; the model paraphrases the already masked or generalized text into natural vague phrasing. Generated candidates are filtered with conservative post-hoc constraints (e.g., suppressing reintroduction of disallowed explicit markers when such a policy is enabled) to reduce degenerate or overtly clinical surface forms.

F.3 Corpus expansion and robustness

The full pipeline can be run in a *multi-strategy* mode that enumerates strategy combinations, yielding many ambiguous counterparts per seed and diversifying the pattern of missing information. An optional lightweight perturbation stage injects small character- and word-level noise into surface strings so that the paired records are not tied to perfectly clean synthetic typography.

F.4 Interface to the clarification agent

Each AIM instance emits a tuple linking an ambiguous user-facing query to metadata naming what was removed or generalized and to the original fully specified query (and, when available, task labels such as diagnosis or safety class). In the present study, the withheld component is exposed only to the simulation oracle that answers clarification questions and to the evaluation scripts; it is never shown to the agent or to the downstream language model. The same tuples can also serve as paired supervision for learned clarification policies, without changing the runtime definition of ambiguity used by the deployed agent.

F.5 Illustrative AIM transformations

Tables 5 and 6 show representative *seed* utterances (fully specified text) and *injected* surface forms produced by individual mechanisms in our implementation. The *Latent* column names the information that remains available as hidden structure (e.g., original query, structured labels, or explicit clarification targets), while the user-facing string is deliberately underspecified. Persona-conditioned rewrites and typo-level perturbations are stochastic; the corresponding rows are marked *illustrative* and show paraphrases consistent with the intended user personas in our generator prompts.

Declaration: use of large language models

In preparing this manuscript, we used large language models (LLMs) to assist with drafting and editing portions of the text for clarity, as well as to support code development. All LLM-generated content was subsequently reviewed, verified, and revised by the authors. No LLM was used to generate scientific conclusions, and all references were manually verified for accuracy. The authors take full and sole responsibility for the accuracy and integrity of all content presented in this paper, in accordance with the journal policy on the use of large language models.

Supplementary Table 5: Mental-health domain: example transformations produced by AIM masking, lexical generalization, and one illustrative vague user paraphrase. *Latent* indicates what is retained as hidden structure and is not shown to the user at inference time.

Mechanism	Seed (specific)	Surface after AIM	Latent
Severity masking	I'm feeling <i>extremely</i> anxious and having panic attacks for 2 weeks. I don't know what to do.	I'm feeling anxious and having panic attacks for 2 weeks. I don't know what to do.	full seed
Temporal masking	I've been depressed <i>for 2 weeks</i> .	I've been depressed for a while.	full seed
Euphemism mapping (risk-preserving vagueness)	I've been having <i>thoughts of hurting myself</i> .	I've been having thoughts. ^a	full seed
Diagnostic phrase removal	I was diagnosed with <i>major depressive disorder</i> .	I was diagnosed with. ^b	full seed
Ontological generalization (domain map + WordNet)	After masking: ... and having <i>panic attacks</i> ...	... and having <i>mental condition</i> ... (domain hypernym map where applicable)	masked intermediate
LLM persona rewrite (illustrative)	(same seed as row 1, after rule-based steps)	"I've been kind of on edge and my body does that thing where it freaks out, it's been going on for a bit."	full seed + strategy tags

^aHigh-risk phrasing is mapped to a vaguer surface form rather than silently deleted, so that the risk cue survives. ^bDiagnostic removal can leave minor grammatical artifacts; in corpus construction such fragments are optionally post-filtered or repaired. Each surface string is paired with the intact seed.

Supplementary Table 6: Food/dietary domain: examples of omission, abstraction, and generalization that force clarification of health context (e.g., diabetes, allergy), amounts, or ingredients. The *Latent* column names the hidden structure that must be recovered before a safety decision.

Mechanism	Seed (specific)	Surface after AIM	Latent
Disease/condition masking	Can I eat white rice with chicken curry? <i>I have type 2 diabetes.</i>	Can I eat white rice with chicken curry?	condition + safety label
Allergen masking	I need a recipe with 2 cups flour and chicken, but <i>I have a peanut allergy.</i>	I need a recipe with 2 cups flour and chicken, but.	allergen + full seed
Quantity removal	Add <i>2 cups</i> of flour to the bowl.	Add some of flour to the bowl.	quantities + full seed
Ingredient omission	I need chicken, onions, and garlic for the stir-fry.	I need onions, and garlic for the stir-fry. (one ingredient dropped)	omitted slots + full seed
Generalization only	I want grilled <i>salmon</i> for dinner.	I want grilled <i>fish</i> for dinner.	full seed
LLM persona rewrite (illustrative)	(masked recipe-style request)	“I want to make that creamy spicy thing, maybe with some meat? I can’t remember what I bought.”	full seed + variables
Surface perturbation (illustrative)	Can I eat oatmeal with honey in the morning?	Can I eat oatmeal with honey in the morning?	same as clean twin